

THE EXTENDED COURSE · SEPTEMBER 2026

From Prompts to Agents

Writing prompts as small work specifications — and delegating work you can check. The written edition of the facilitated course, complete with the five worked tasks: analyze the data, build a dashboard, present the findings, summarize the thread, and explain it clearly.

THIS BOOK	The course, cover to cover, for self-study or review after the live session.
THE DECK	From_Prompts_to_Agents_Facilitated_60_Minute.pptx — the 60-minute live course.
THE WORKBOOK	Course_Workbook.xlsx — the data and the five task tabs this book works through.
THE BUILDER	Prompt_Template_Creator.html — assembles prompts from the same requirement menus.
ONE PAGE	From_Prompts_to_Agents_Cheat_Sheet.pdf — the skeleton and the menus, printable.

All names, companies, and numbers in this book are fictional and generated for practice. Your organization's AI policy and approved-tool list outrank anything written here.

CONTENTS

Foundations: how these systems work	4
One skill, two ways to work	4
Four eras, and the decade of named leaps	4
How a language model produces an answer	5
Tokens and practical limits	6
Context, saved history, and memory	6
Answers grounded in documents	7
Effort, capability, and the smallest fix	8
Verification belongs in the task	8
Failure patterns and the check they need	9
Models and tools: choosing where the work happens	11
Compare on your task, not on the leaderboard	11
January 2025: the efficiency moment	11
The right workspace for the data	12
Prompt engineering: writing the work specification	14
You already write work specifications	14
Good requirements make prompts testable	14
A prompt is a small work specification	15
Useful roles describe behavior	16
Prompt elements are requirement types: the menu	16
Examples, uncertainty, and stopping	17
A menu of requirements for each artifact	18
The improvement loop	19
One task through the improvement loop	19
Prompt techniques to test	20
A neutral rubric reduces leading feedback	21
Optional practice: a course-log study aid	22
Putting prompts to work: one case, five artifacts	23
One question, five artifacts	23
Task 1: analyze spreadsheet data	24
Task 2: build a dashboard you can check	27
Task 3: present the findings	29
Task 4: summarize the email conversation	31
Task 5: explain a topic clearly	34

The arc, complete	35
Agentic work: delegating with boundaries	37
What makes a workflow agentic	37
ASK and DELEGATE: the decision, not the tools	37
Which way of working fits: a self-test	38
The anatomy, grown up to survive autonomy	38
The mission brief for bounded work	39
Worked exercise: the supplier analysis becomes a mission brief	40
Choosing an environment for delegated work	41
Instructions versus enforceable controls	41
Diagnosing a failed run	42
Write the brief once — the workspace remembers	42
Exercise: one gate and one hard check	43
Reusable prompts: from one good result to a shared asset	44
A useful prompt becomes a reusable asset	44
Where reusable prompts can live	44
A small review process for shared prompts	45
Save one prompt for Monday	46
Let the assistant interview you	47
The habits to carry into work	47
The capstone: one task, one checked result	48
Your next workplace application	48
Reference: the glossary, the criteria, and the evidence	50
The glossary: twenty terms that matter	50
Why each dial matters: the generative criteria	51
Why each dial matters: the agentic criteria	51
The evidence box set: works, myth, expired	52
Sources this training is built on	54
The package	55

PART 1 - FOUNDATIONS

Foundations: how these systems work

Everything in this course rests on a handful of mechanisms: what a language model actually does when it answers, what it can and cannot remember, and why a fluent answer is not the same thing as a true one. This chapter builds that foundation from zero. Read it with an assistant open - several sections end with a prompt to run, copied from a named tab of Course_Workbook.xlsx in the deliverables folder.

One skill, two ways to work

Prompting is the one AI skill that transfers everywhere: every assistant, every vendor, every year. Models change monthly; the craft compounds. The skill has two modes. In **ASK** mode you request an answer or a draft, read it, check it, and decide what happens next. In **DELEGATE** mode you assign bounded work: you specify the inputs, checks, permissions, and finished result, and the system plans, uses tools, checks itself, and delivers.

The ten-second version: a generative prompt describes what to write; an agentic prompt describes a job to run. That distinction is the backbone of this course - the craft chapters teach the first kind, the agentic chapters grow it into the second. For now, feel the difference.

TRY IT — Open your course log

Open a fresh chat and keep it all course - your course log. Run the two steps below, from tab EX1-TwoModes of Course_Workbook.xlsx.

STEP 1: Write a short farewell card for a coworker who is leaving.

STEP 2: Now don't write it. Ask me everything you'd need to know to do this perfectly, then wait for my answers.

Tab EX1-TwoModes - run both steps in one chat, and keep that chat.

Step one is ASK: instant text, every unknown guessed. Step two is the seed of DELEGATE: the questions come to you. If your assistant asks questions in step one too, that is no failure - asking is the delegate move; step two makes it happen on purpose.

Four eras, and the decade of named leaps

Seventy years of AI compress into four eras. In the **rules** era (1950s-1990s), humans hand-coded the logic: if X, then Y. Expert systems did real work - MYCIN diagnosed infections, XCON configured computers - but rules crack on messy reality, and hype outran results twice, in the funding collapses remembered as the AI winters: skepticism is historically earned. In the **learning** era (1990s-2010s) the approach inverted: let the system find patterns in examples. It gave us spam filters, search ranking, recommendations, and - decisively in 2012, when deep networks on gaming GPUs cut image-recognition error from 26.2% to 15.3% (AlexNet) - modern image recognition.

The **generative** era began in 2017 with the Transformer, which lets models learn patterns from large datasets: predict-the-next-word at that scale writes, summarizes, and codes, and ChatGPT (2022) made it something everyone could talk to. The **agentic** era (2024 onward) gives models arms - tools, browsers, files, code - and reasoning models that plan multi-step work and execute it

with limited supervision. AI you delegate to.

KEY — Why the last decade exploded

Scaling laws (2020): a bigger model, more data, and more compute produced improved measured performance under the studied conditions. Progress stopped being a research gamble and became an investment roadmap.

Year	The leap	What it changed
2017	The Transformer (Attention Is All You Need)	A foundation for many subsequent language models
2020	Scaling laws + few-shot (GPT-3)	Show examples in the prompt and it does the task - prompt engineering is born
2022	Instruction tuning + RLHF	Human feedback turns a predictor into an assistant; ChatGPT reaches 100M users in two months
2024	Reasoning models	Compute spent at answer time, not just in training
2025	MoE, distillation, open weights	Frontier ability gets radically cheaper; big models teach small ones
2026	Agentic harnesses	Delegation reaches general office work

Each method on this list returns later in the course - treat it as vocabulary.

Adoption outran precedent: ChatGPT reached 100 million users in about two months - the fastest consumer ramp on record at the time (UBS/Reuters, 2023), against roughly nine for TikTok and thirty for Instagram. Access changed faster than workplace processes adapted; that gap is why this course exists.

The hardware, in scoped terms

Two principles are enough. **Training:** large models learn from repeated computation over data, so parallel processors, memory, and data movement constrain what can be trained. **Serving:** each request also requires computation, so model design, requested reasoning effort, and workload drive response time and operating cost. Reading hardware claims, keep three categories separate: measured results, estimates, and announced investments.

How a language model produces an answer

The useful mental model: a prediction engine, sent to finishing school. In **pretraining**, the model spends months on trillions of words learning one task - predict the next token - and language, facts, and reasoning patterns compress into billions of learned weights. **Instruction tuning** uses curated example dialogues to teach it to follow instructions rather than merely continue your text. **Human feedback** (RLHF) then tunes it toward helpful, honest, and harmless, from people ranking outputs. How your own feedback and chats are used depends on the product, account, and data settings - check, don't assume.

Two consequences follow. First, knowledge is frozen at the training cutoff, months before release - which is why assistants bolt on escape hatches: live web search pulls today's pages into the prompt, and retrieval plugs in your own documents (below). Second, the model completes your text. The prompt is one way to steer the work, alongside sources, tools, and settings - and a complete prompt

does not make the model smarter. It deletes wrong guesses.

KEY — Prediction, not verification

A language model predicts useful continuations; it does not independently verify truth. A fluent answer can still contain an unsupported claim.

SEND 1: Finish this sentence 5 different ways: We should move the launch date because

SEND 2: Now 5 more ways, knowing: B2B software firm, competitor launches May 3, our beta ends April 20.

Tab EX2-Guesses - two separate sends, in your course log.

The first send scatters across plausible guesses - budget, staffing, quality. The second orbits May 3 and April 20. Grade the directions, not the sentences: your facts deleted wrong guesses. That is your contribution to every answer: the task, the source material, the definitions, and a way to check the result. Compare 'Explain the delay' with 'Explain the delay using these three dated messages' - only the second can be checked. A model filling unstated requirements with its most plausible guess matches what the requester meant only about 41% of the time (Yang, 2026); this course is about closing that gap on purpose.

Tokens and practical limits

The model never sees letters. Text is chopped into **tokens** - word chunks from a fixed vocabulary, like bricks of text. Common words are one pre-molded brick; rare words get built from pieces (ham-bur-ger). English rules of thumb: one token is about four characters, or three-quarters of a word; a 50-page document runs roughly 25-35 thousand tokens. A token is not a fixed number of words, and limits vary by model and application - check your workspace's actual limits.

Three practical consequences. Everything is priced and limited in tokens - and output typically costs around five times input (as of August 2026), because generation is serial - so paste-heavy prompts burn budget and context fast. Non-English and dense technical text cost more tokens for the same meaning. And character-level precision is not the model's native ability: use AI for language and software for characters - counts, checksums, exact IDs.

EXPIRED — The strawberry test

The famous letter-counting failures of 2024 (miscounting the r's in 'strawberry') are patched on current frontier models as of September 2026. Still breaking: exact word counts and counting their own tokens on every tier; character-precise edits on fast tiers. For exact checks, count with a tool, not the model.

Explain AI tokens to a high-school student in under 80 words: use a LEGO-brick analogy, show one word splitting into tokens, and end with why tokens set my AI's cost and limits.

Tab EX3-Tokens - the definition lands in your course log, from your own tool.

Context, saved history, and memory

Three things sound alike and are not. **Active context** is the information available to the model for this response. **Saved chat history** is a stored conversation you may be able to reopen - closing a window does not necessarily delete it. **Cross-chat memory** is a product feature that may carry selected information into other conversations. Only the first is guaranteed to influence the answer in front of you.

For active context, use the desk analogy: the context window is working memory for one conversation. Your question, the system prompt, every prior turn, attached files, and tool results must all fit on the desk. By 2026, roughly one million tokens (about 750,000 words) is the flagship standard - but consumer apps often enforce smaller limits, and a model's published limit may differ from what your interface makes available.

WORKS — Lost in the middle

In long contexts, accuracy is highest at the start and end and sags in the middle (Liu et al., 2023). Placement alone is worth up to ~30% on long documents (Anthropic's long-context guidance).

- Long documents at the top, the question at the end - check the effect on your own task.
- Label multiple documents so the model can tell them apart.
- New topic, new chat - old turns are still on the desk, steering.
- Long thread? Run the handoff move below.

Just as important is what the model does not remember. Nothing persists into a fresh conversation unless a memory feature or a saved file reloads it, and a very long chat may compress or omit earlier material. Carry important requirements forward yourself: for a handoff, save the goal, sources, decisions, open items, and next step.

STEP 1: Explain your context window like I'm a 5th grader: the desk, what fits on it, and what happens when I close this chat. Under 100 words.

STEP 2: Summarize our chat so far in under 80 words, titled HANDOFF.

Tab EX4-Handoff - later, paste the HANDOFF into a fresh chat and ask 'where was I?'

One term to file for the agentic chapters: **context engineering**. The prompt is the briefing; context is everything on the desk, including things nobody typed - tool results, standing instructions. Agentic tools bolt a filing cabinet onto the desk: persistent files and notes that outlive one conversation.

Answers grounded in documents

The model's built-in knowledge is frozen and public; your work runs on private documents it has never seen. **Retrieval-augmented generation** (RAG) bridges that gap at answer time - an open-book exam. You ask a plain question ('What does the supplier agreement say about re-inspection?'); a retrieval step finds the most relevant passages - semantic search, by meaning rather than keywords; those passages land on the model's desk behind the scenes; and the model answers from the supplied text, citing where it looked.

Grounding beats recall for three reasons. It is **current**: documents are re-indexed in minutes, with no retraining. It is **checkable**: citations point to the exact passage, which free recall can never do. And it can respect boundaries - though privacy and access depend on the actual service, its permissions, retention, and training-use settings, so confirm what your organization runs. Retrieval feeding one answer is not training the model on your documents; that is the privacy point worth repeating. Many internal document assistants work exactly this way.

TRAP — The wrong page, beautifully cited

Fetch the wrong page - an outdated revision, a near-miss document - and the model still writes a fluent, confident, cited answer from it. An outdated agreement can support a well-cited but wrong answer for today's decision.

CHECK — Audit the librarian

Ask for the document, the revision, and the supporting passage. Then read the cited passage and confirm it actually answers the question. Check the citation, not just the prose.

Effort, capability, and the smallest fix

Instructions, sources, and model adaptation

When an answer disappoints, there are three sizes of intervention, and the smallest that solves the problem wins. Clearer **instructions** fix behavior - tone, format, focus, approach. Better **source material** fixes missing knowledge - your policies, your data, current facts; most real workplace cases end here. **Model adaptation** (retraining) changes deep habits permanently; it is slow, expensive, and rare - an IT project, justified only by a recurring need that evaluation shows simpler changes cannot meet. One question picks the level: is the model missing knowledge (supply documents), or misbehaving with knowledge it has (fix the prompt)? More documents or more training do not automatically solve unclear requirements.

Reasoning effort and the task

Assistants ship a fast path - one pass, instant, cheap - and a thinking path that drafts, checks, and revises internally before answering. The model is not literally thinking; it generates intermediate tokens that improve the final answer, and those are billed like output, the expensive kind - a hard question can quietly cost 5-20 times a simple one. Try more effort for a difficult comparison, several dependent calculations, or a result requiring multiple checks; keep it off for lookups, reformatting, summaries, routine drafting. Measure correctness, useful evidence, response time, and cost in the account you actually use, and escalate on failure, not by default - more computation does not replace clear requirements or verification.

Capabilities to look for in an assistant

- Quick response - when the task is clear and the output is easy to check.
- Additional reasoning - for a difficult comparison or a task requiring several checks.
- Search or document retrieval - when the answer needs current or supplied source material.
- File creation and tool use - when the deliverable includes calculations, editable files, or actions.

Product menus change, and names, limits, and costs vary by vendor and account - so verify current documentation and your account's actual access rather than memorizing a catalog, and never rely on a model's description of where its own buttons live. The durable habit: pick the smallest mode that can succeed, and escalate deliberately.

List the modes this app gives me - quick answer, thinking, web search, deep research, agent - one line each on what it does differently and its rough effort. Say 'unsure' rather than guess.

Tab EX6-FeatureMenu - self-updating: ask again the day a button changes.

Verification belongs in the task

Now the mechanism that makes verification non-optional. Models are optimized for the most plausible next token, not the most true one, and trouble concentrates where training data is thin: rare facts, citations, numbers, names. OpenAI's own 2025 research adds the incentive: benchmarks reward confident guessing over 'I don't know', so models learn to be good test-takers. The better

word than hallucination is **confabulation** - gaps filled with plausible material, in format: fake citations look exactly like citations.

The effect is measurable. In the comparison OpenAI published in 2025, an older reasoning model answered 24% of hard factual questions right and 75% wrong; a newer model, trained to say 'I don't know', answered 22% right and 26% wrong, abstaining 52% of the time - roughly three times fewer false claims at nearly the same amount right. The confident guess is a training artifact; explicit permission to not know - the move the craft chapters call the Out - recovers the honesty the benchmarks trained away.

MYTH — 'Are you sure?' is not verification

Pushback is a social signal, not a re-derivation: the model re-reads you, not its work. Answers flip in 58% of cases, 14.7% from right to wrong (SycEval, AIES 2025). Verification is a named check or an outside source - never the same model reassuring you.

So verification belongs in the task itself. 'The supplier approved the change' invites one question: what message supports that claim? 'Approval is conditional on the cost cap' - with the message shown and the condition preserved - can be checked, and keeps the qualifier that makes it true. When you set a task, name the source or calculation that would settle the question.

KEY — The rule of three

For numbers, recompute. For claims, inspect the source. For actions, inspect the result.

The public record shows what skipping the check costs. In June 2023 a New York lawyer filed six cases that did not exist - then asked ChatGPT whether they were real, and it said yes (\$5,000 sanction, *Mata v. Avianca*): he checked the fabrication with the fabricator. In February 2024 a tribunal made Air Canada honor a refund policy its website chatbot had invented - your organization owns what its chatbot says. In May 2024 an AI search feature served a joke forum comment as advice: glue in pizza sauce. Across 2025-26, three global consultancies had reports challenged over invented or failed citations, and a legal tracker counted about 1,500 court decisions worldwide involving AI-fabricated citations by mid-2026. Each time, the missing thing was a check someone could have run.

The cure has not changed: **ground it** (give it the source), **cite it** (demand receipts, plus an 'I don't know' escape hatch), **verify it** (check what matters before it ships). None of these eliminates confabulation - they convert unverifiable claims into verifiable ones. House rule from here on: every uncited standard clause, date, number, or quote is a draft until verified.

Failure patterns and the check they need

The last foundation is diagnostic. When an answer is wrong, the reflex is to reword the whole prompt and hope. Resist it: output quality swings up to 76 accuracy points from formatting alone (Sclar et al., ICLR 2024), so one failed phrasing is a data point about the prompt, not the capability ceiling. The productive move is to name what kind of wrong you got, because each kind points at the part of the prompt that owns it - the elements the craft chapters build in full. The grid below is the map the course keeps returning to.

The output is...	The element that failed
The wrong altitude, tone, or posture	Role

The output is...	The element that failed
An answer to a different (or vaguer) question	Task
Generically right, specifically wrong for us	Context
Right content, unusable shape or length	Format
Not matching the standard in your head	Examples
Confidently invented	The Out (missing)
Sprawling past what you asked	The Stop (missing)

The diagnosis grid - also on page 1 of references/Elements_of_Prompting_Field_Guide.pdf.

Two rows deserve a word now. 'Generically right, specifically wrong' means the model filled your specifics with the internet's average - the Context row. And the two 'missing' rows are the safety valves: the Out is permission to say 'I don't know'; the Stop is the boundary that turns open-ended capability into bounded work. The rest you will recognize the moment they happen to you.

The same discipline applies to any answer you are about to rely on. Four patterns recur, each with its check: an **invented fact** - find the supporting record, or state that it is unknown; an **invented citation** - open the actual source and inspect the cited passage; the **wrong document or revision** - confirm identity, date, and applicability first; a **lost condition** - compare the conclusion with the original qualifier or approval condition.

CHECK — Format is not evidence

A credible format, confident tone, or citation marker is not a substitute for checking. A memorable failure story matters only if it changes which checks you run.

That completes the foundations: two ways of working, a prediction engine steered by what you supply, hard limits on memory and attention, grounding that beats recall, effort dialed to the task, and verification designed into the work. Next comes the craft: turning what you already know about writing requirements into testable prompts - practiced on one fictional supplier case that runs through the whole book.

PART 2 - MODELS AND TOOLS

Models and tools: choosing where the work happens

Before you invest in the craft of prompting, spend a few minutes on two decisions that sit upstream of every prompt: which assistant does the work, and which workspace is allowed to see the data. The assistant market re-ranks monthly, so this chapter deliberately teaches you a method for choosing rather than a ranking to memorize. The names churn; the habits you leave with do not.

Compare on your task, not on the leaderboard

Public leaderboards answer a question you are not asking: which model scores highest on someone else's benchmark. Your question is narrower and more useful: which assistant, on the account you actually have, does your task well. The course materials date every product characterization as of September 2026 precisely because such claims expire; the comparison method below does not.

When you need to choose an assistant for a piece of work, walk four questions in order:

- **Required output.** Can this account produce the text, calculation, file, or action you need?
- **Source access.** Can it use the right documents and show supporting evidence?
- **Work environment.** Does it fit the approved workspace and collaboration process?
- **Evaluation.** Run the same representative task on the candidates and compare correctness, effort, and cost.

The assistants you will meet by name in this course include ChatGPT, Claude, Copilot, Gemini, Perplexity, and Grok; verify current offerings before relying on any characterization of them. What stays true across the churn is that different assistants expose different capabilities and controls, and the same brand behaves differently on different account tiers. That is why the fourth question, a small head-to-head trial on a task that represents your real work, settles more arguments than any review article.

KEY — The rule that survives the churn

Tool choice follows the job, the evidence, and the approved data boundary, not habit and not a leaderboard.

EXPIRED — Rankings age in months

Any model ranking or product claim without a date is expired on arrival. Treat dated names and rankings as reference context, not a purchasing guide, and re-verify before a decision that matters.

TRY IT — A ten-minute bake-off

Pick one task that represents your real work. Run the identical prompt in two assistants you can access, then compare correctness, effort, and cost. Keep the result with its date.

January 2025: the efficiency moment

One event explains why the advanced capabilities from Part 1, extended thinking and deep research, did not stay luxury-priced. In January 2025, the Chinese lab DeepSeek released R1, and three facts made it a landmark:

- DeepSeek-R1 was an open-weights reasoning model rivaling OpenAI's o1, with MIT-licensed weights anyone could download.
- Nvidia lost \$589 billion in market value in one day (January 27), then the largest single-day market loss ever.
- The famous "\$5.6M training cost" headline described real efficiency but was oversold: that figure was the fuel for the final race, not the cost of the racing team.

The trick has a name: Mixture of Experts

Picture a huge specialist hospital at night. The building holds 671 billion parameters on the books, but for any one question only about 37 billion wake up: the lit windows, the relevant departments. You pay for the specialists consulted, not for the whole building. Distillation, big models teaching small ones, finishes the job. The result on the bill, per million output tokens as of January 2025: OpenAI's o1 cost \$60, DeepSeek-R1 cost \$2.19, roughly 27x cheaper. A \$100 o1 workload cost about \$3.60 on R1.

Benchmark	DeepSeek-R1	OpenAI o1
Math olympiad (AIME 2024)	79.8	79.2
Coding agent (SWE-bench)	49.2	48.9
Science PhD (GPQA)	71.5	75.7

Scores as reported in the R1 paper, January 2025. Matched on math and coding, behind on PhD-level science: not a clean sweep.

TRAP — The oversold headline

Quoting "\$5.6M to train a frontier model" without its scope repeats a distortion: the number covered one final training run, not the hardware and research behind it. Real efficiency gain, wrong denominator.

KEY — Why this belongs in your reference layer

Vendor and pricing history is context, not current guidance. Costs measured on different dates and by different methods are not directly interchangeable; always carry the date with the number.

Explain Mixture of Experts like a colleague: a specialist hospital where only the relevant departments wake up per question – and why that made AI dramatically cheaper in 2025. Under 120 words.

Copy from tab EX7-MoE of Course_Workbook.xlsx; the answer becomes the MoE cheat-note in your course log.

The right workspace for the data

Capability is the second question. The first is where the data may go. Data sensitivity decides the tool, and the decision runs through three checks before anything is uploaded:

- **Approved account.** Use the service and the account approved for this work.
- **Allowed material.** Confirm what may be uploaded, retained, and shared.
- **Minimum access.** Give the tool only the files and permissions the task requires.

A vendor's home country alone does not describe every customer's storage arrangement. Company data rules depend on the actual account, service, contract, and configuration. Data residency, choosing where your data lives, is typically an enterprise feature: the same vendor can host your company tenant in the EU while its consumer app stores chats in the United States. Same logo, different homes; the account tier decides which one is yours. The one question that settles it is for your IT department: which region and tier does your account run on? Your organization's AI policy answers everything else.

A distinction worth keeping when open-weights models come up: using a vendor's app is eating at their restaurant, where they see your order. A downloaded open-weights model is a cookbook your IT team can cook from at home. The app, not the model, is where the data-handling risk lives.

This is also why every exercise in this course runs on fictional data: the supplier quality scenario with Alpha Components, Bravo Plastics, and Cardinal Metals in the Data tab of Course_Workbook.xlsx. It behaves like real operational data but is safe in any workspace, so you can practice the craft before you ever face the upload question with real records.

KEY — The standing rule

Your organization's AI policy and approved-tool list outrank every product named in this part, and they outrank this book. Where they conflict with anything written here, the policy wins.

TRAP — Judging by the logo

Inferring where your words go from a vendor's headquarters or brand reputation skips the variable that actually matters: the tier and configuration of the account in front of you.

CHECK — Before any upload

Name the approved account, the allowed material, and the minimum access for this task. If any of the three is uncertain, resolve it through your organization's approved guidance before uploading, not after.

PART 3 - PROMPT ENGINEERING

Prompt engineering: writing the work specification

This is the craft chapter: how to get what you actually meant. Its central claim, once seen, is hard to unsee: writing a good prompt is writing a good requirement — and you almost certainly write requirements already. The chapter builds the anatomy of a strong prompt, walks the menu of requirement types, and puts both on an improvement loop.

You already write work specifications

Start with a recognition. Requirements, standard operating procedures, and test protocols exercise one core skill: stating what a piece of work must contain, how it must behave, and how you will know it is right. A prompt asks the same of a reply or an artifact — not a different skill, a different audience. Andrew Stellman, 2025: "Prompt engineering and requirements engineering are literally the same skill — using clarity, context, and intentionality to communicate your intent."

The forms map directly. A user story — "As a [role], I want [goal], so that [benefit]" — is role plus task with a purpose clause. Given [state], When [action], Then [result] maps to context, task, and format plus checks. The ISO 29148 characteristics — unambiguous, complete, singular, verifiable — are what this chapter asks of every prompt line.

The stakes: a model guesses an unstated requirement right only about 41 percent of the time (Yang, 2026); a written specification deletes the guessing. Software moved first — GitHub's Spec Kit (2025) makes the written spec "the source of truth" AI agents build from, and Sean Grove of OpenAI drew the conclusion: "Whoever writes the spec... is now the programmer." One caution: this evidence is structural, measured on clarified prompts — and a model can still misinterpret a clear instruction, so verification stays necessary.

KEY — The hardest 80 percent, already owned

If you can write a procedure, a test protocol, or an acceptance criterion, you own the hardest part of prompt engineering. The elements are a requirements document in miniature.

Good requirements make prompts testable

Two ideas carry this chapter. **Requirement type** names what must be specified — role, context, format, method. **Requirement quality** describes how well any selected requirement is written. The quality bar applies the writing principles summarized by NASA (Systems Engineering Handbook, Appendix C) and INCOSE (Guide to Writing Requirements).

Quality	For a prompt, it means	The failure it prevents
Necessary	Every line earns its place; a simple task needs a few.	Rule piles that bury what matters.
Clear	One reading; named columns, defined terms, no "etc."	Ambiguity resolved silently, the model's way.

Quality	For a prompt, it means	The failure it prevents
Complete enough	The consequential facts and rules stated — not every fact.	Gaps filled with plausible guesses.
Consistent	No requirement contradicts another.	The model reconciling you with yourself.
Feasible	Possible with the data and tools supplied.	Confident output the data cannot support.
Singular	One requirement per line — separable, revisable.	Half-followed compound instructions.
Verifiable	An observable check exists: a number to reconcile, a behavior to exercise.	"Looks done" standing in for done.

A requirement you cannot check is a wish. Separate the required outcome from a preferred implementation.

For a data prompt, "complete enough" means: dataset and period, row boundaries, calculation definition, deliverable, missing-data treatment, reconciliation. "Tell me which supplier is worst" leaves the model to pick the rows, the metric, and the meaning of "worst", silently; "calculate returns divided by units shipped by supplier and reconcile to the source" carries its own test.

CHECK — Verification and validation

Verification: does the output meet the written criteria? Validation: does it serve the actual business need? Check both, every time — these principles improve clarity and testability without making model behavior deterministic.

A prompt is a small work specification

One worked case: a manager must choose supplier follow-up. The prompt — calculate return rate from the supplied workbook, as total returns divided by total units shipped; exclude the existing TOTAL row; return a ranked table with a brief, neutral explanation; state missing information; reconcile totals to the source. Each sentence is a requirement: audience, source, method, format, tone, check. The exclusion prevents double counting; the reconciliation makes the answer checkable. Generalized, every prompt shares one skeleton:

Purpose & audience: help [reader] make [decision] or complete [task].
 Sources & scope: use [source / version / period]; include [scope]; exclude [items].
 Selected requirements: only the types that resolve ambiguity for THIS artifact — a simple task needs a few, not all.
 Acceptance evidence: check [specific result] against [source, calculation, or observable test].
 Missing info & boundaries: if [gap or conflict]: state it, ask, or stop.
 Actions needing review: [actions].

The skeleton of every prompt — printed on the cheat sheet ([deliverables/From_Prompts_to_Agents_Cheat_Sheet.pdf](#)).

The course's fuller anatomy has seven parts: five elements — Role, Task, Context, Format, Examples — plus two one-sentence safety valves, the Out and the Stop. Written out on a mundane task, the whole body is one continuous prompt of about ninety words, under two minutes to write:

ROLE You are a precise editor. You add no facts that are not in the source.
 TASK Summarize the attached Q2 returns report for the operations lead.
 CONTEXT We're a mid-size electronics manufacturer. Return rate = returns ÷ shipped.
 The export has a totals row – exclude it.
 FORMAT ≤ 150 words: the headline number first, then three bullets.
 EXAMPLES <example> March's summary – the one the team liked </example>
 THE OUT Anything the report doesn't state: write UNKNOWN – never estimate.
 List the gaps at the end.
 THE STOP Deliver the summary, then stop – nothing beyond.

Seven parts, one body. Miss a part and the prompt still works — it just fails in a predictable way, which is what makes diagnosis possible.

Notice the industry line: what business you are in is context — the assistant does not know your world until you say so. And note the Out's formal definition: the escape route you write for missing information — a condition, an honest move, a surfacing — so the model never has to choose between obeying you and being honest. Anthropic's golden rule applies to the whole body: show the prompt to a colleague with minimal context; if they would be confused, the model will be too.

Useful roles describe behavior

"Act as a senior analyst" is a title — a vibe the model can fake. "State your assumptions, calculate from the source, and identify what the data cannot answer" is conduct a reader can observe: either the assumptions were stated or not. The craft is decomposing an identity into personality statements — testable behaviors, not credentials — paired with a concrete question and an intended audience.

The evidence draws the boundary. Expert personas do not improve factual accuracy: 162 personas, no gain (Zheng et al., 2024), confirmed by a six-model replication (Wharton, December 2025). What a role reliably shifts is voice and register — assigned traits are identifiable in output up to 80 percent of the time (PersonaLLM, 2024). Use Role for how it answers, never instead of checking what it answers.

- Weak: "You are a senior analyst." Strong: "You never invent numbers; you state n; you say what the data can't answer."
- Weak: "You are a helpful assistant." Strong: "You challenge weak assumptions instead of agreeing; you flag uncertainty rather than smooth it over."

KEY — Behaviors beat titles

A role earns its place by naming checkable conduct, not by conferring rank — each personality statement is auditable in the output; a title is a vibe.

Prompt elements are requirement types: the menu

Role maps to behavior, context to scope, tone to communication, format to structure — and the mapping keeps going. The menu below names fourteen requirement types; you pick from it, you never fill it. Each type carries acceptance evidence — how to check compliance — in the requirements catalog, printed in full in references/Elements_of_Prompting_Field_Guide.pdf.

What the work is and who it serves

- **Functional behavior** — what it must do: filter, calculate, extract, compare, explain. Weak: "analyze the returns data." Strong: "calculate the return rate per supplier and rank all three."

- **Scope and exclusions** — in, out, explicitly outside. Weak: "use the relevant data." Strong: "detail rows only — exclude the TOTAL row; keep defects separate from returns."
- **Audience and use** — who uses it, for which decision. Weak: "make it useful for leadership." Strong: "for a manager deciding whether supplier follow-up is warranted."
- **Role and working behavior** — perspective plus observable conduct. Weak: "act as a reviewer." Strong: "act as a reviewer: expose assumptions and gaps."

What goes in and how it is computed

- **Data and provenance** — allowed inputs, definitions, period, traceability. Weak: "use the supplier data." Strong: "this workbook's Data tab, 2025-09 to 2026-08 inclusive; trace each figure to its rows."
- **Method and business rules** — formulas, denominators, missing-data treatment. Weak: "use our standard definitions." Strong: "return rate = total returns ÷ total units shipped, from sums, never averaged percentages; the 'n/a' cell is missing, not zero."
- **Content and completeness** — required topics, fields, limitations. Weak: "cover the important points." Strong: "the business question, the comparison, one time view, a follow-up, limitations."

How the output looks and behaves

- **Structure and interface** — sections, tabs, slides, columns, controls. Weak: "make it a nice table." Strong: "one row per supplier: Supplier | Return rate | Numerator | Denominator — highest rate first."
- **Tone and presentation** — voice, reading level, units, number formats. Weak: "keep it short and professional." Strong: "neutral, plain language for managers; rates to three decimal places."
- **Accessibility and usability** — labels, contrast, keyboard access, understandable language. Weak: "make it user-friendly." Strong: "labeled controls, visible focus, readable contrast, a usable narrow layout."

How the result is checked, bounded, and kept

- **Quality and acceptance** — observable correctness criteria. Weak: "double-check your work." Strong: "reconcile total shipped and returned against the TOTAL row before presenting anything."
- **Constraints and authority** — allowed tools, approvals, stop conditions. Weak: "be careful with the emails." Strong: "prepare the reply as a draft for review; do not send it." (Prompt instructions request compliance; actual permissions determine what a tool can do.)
- **Delivery and compatibility** — file type, editability, offline use, destination. Weak: "send me a dashboard." Strong: "one self-contained HTML file that works offline, data embedded, no network calls."
- **Maintenance and reuse** — refresh procedure, versions, reusable parameters. Weak: "so we can use it again." Strong: "label the file a static data snapshot with its date; keep reusable placeholders."

KEY — Pick, don't fill

The menu is a lookup aid, not a form every prompt must complete. Select the types that resolve ambiguity in this task, write each well, and define how you will check it.

Examples, uncertainty, and stopping

Examples are the strongest steering signal for format and tone: the model imitates what it sees, and on well-tuned models examples steer shape more than facts (Brown et al., 2020; Min et al., 2022). The discipline is diversity: three clones of the happy case teach only the center; one typical, one edge, and one reject case — clearly separated — teach the boundaries too. Use an edge case when it exposes a likely failure, and never smuggle obsolete facts in with the sample.

The Out is permission to not know, in checkable shape. A model is trained to always produce an answer; unless you name the honest move, inventing is the only way it can obey you. One sentence of permission cut invented answers on planted-error questions from 53 to 23 percent on GPT-4o (six-model average 66 to 44 — Omar et al., 2025), and to under 8 percent on GPT-5 (npj Digital Medicine, 2026). "Write Not stated and list the gap" is the pattern — a named form you can look for in the output.

TRAP — Bare bans backfire

"Do not hallucinate" names no honest alternative, and a ban alone can trigger over-refusal of facts that are in the supplied material. Say what to do instead: the Out, plus required citations — it helps without guaranteeing that nothing is invented.

The Stop is the finish line: the action to perform and its boundary. Weak: "keep it focused." Strong: "answer these three questions, report limitations, then stop — do not go exploring." A completion condition converts open-ended capability into a scoped job — instead of ten charts and no answer — and in agentic work this valve grows up to become gates and autonomy rules.

KEY — The unifying idea

Each element moves something out of your head onto the page. When output disappoints, one of them stayed in your head — and the diagnosis grid below tells you which.

A menu of requirements for each artifact

The artifact decides which requirement types matter most. The working method: recognize the artifact, choose the types that resolve ambiguity in this task, then write a concrete instruction and identify the evidence that would show it was met.

- **Spreadsheet analysis** — calculation rules: row grain, period, metric definitions, denominators, missing values, reconciliation.
- **Interactive dashboards** — behavior and state: date ranges, filters, coordinated views, empty and zero states, Reset, offline portability.
- **Presentations** — narrative and editability: audience, story structure, one message per slide, data fidelity, speaker notes.
- **Email briefs** — chronology and evidence: current versus superseded positions, decisions with conditions, owners and dates, the draft-only boundary.
- **Plain-language documents** — reading level and fidelity: vocabulary, structure, analogy limits, preserved conditions, a comprehension check.

The per-artifact tables — requirement type, example instruction, how to check it — are printed in the field guide; the same menus drive the Prompt Template Creator ([deliverables/Prompt_Template_Creator.html](#)) and the Configurator ([references/Prompt_Template_Configurator.xlsx](#)). Treat it as a catalog: something disappointed, name the weak spot, pick a stronger fill. You do not study it; you look things up in it.

The improvement loop

A first prompt is a draft, and the way from draft to dependable is a loop you may already run on processes: Plan, Do, Check, Act — the improvement cycle of W. Edwards Deming, which Toyota built its production system around. The Lean Enterprise Institute published a piece in May 2026 applying it to prompting as "Prompt, Do, Check, Act"; their observation: the people who get the most from these tools are the ones willing to run the loop a few more times. Every phase can be partly delegated while judgment stays with you.

- **Plan — draft the ask.** Fill the elements the task needs; say why behind each rule; separate instructions from pasted material; show examples, one edge case included. Delegate it: "I need [X] for [audience], with [constraints]. Draft me a strong prompt, then wait."
- **Do — run it, in order.** Number the steps; add "wait for my OK" where you want control; documents on top, ask at the end; fresh chat per topic.
- **Check — inspect before you trust.** Name the failed element with the grid below; self-check against named criteria — never "are you sure?"; blind-review by pasting the draft as "a colleague's"; reconcile numbers to a total you already trust.
- **Act — refine, then standardize.** Fix one element and rerun the affected check — never reword at random; when a prompt passes representative checks, name it, version it, save it; re-baseline on every model upgrade. Delegate it: "Rewrite this prompt so it more consistently produces X."

The output is...	The element that failed
the wrong altitude, tone, or posture	Role
an answer to a different (or vaguer) question	Task
generically right, specifically wrong for us	Context
right content, unusable shape or length	Format
not matching the standard in your head	Examples
confidently invented	The Out (missing)
sprawling past what you asked	The Stop (missing)

The symptom-to-element diagnosis grid — also in the EX-Report tab of deliverables/Course_Workbook.xlsx. Name the failed element, fix that one, rerun.

Keep a change only when it improves the result on representative examples, not one lucky run. Model self-review helps find issues, but it is not proof — the checks that count compare the output with evidence or a known calculation.

One task through the improvement loop

The loop at its smallest, on the supplier case. Initial ask: "Summarize supplier returns." Observed problem: the total includes a pre-calculated TOTAL row — units counted twice. Revision, aimed at that failure: "Use detail rows only; exclude the TOTAL row." Verification: compare with the source total — an independent check only when scope, filters, and units match. Change the instruction linked to the failure; keep the business question stable. A passing test supports reuse; it does not establish universal reliability.

Run enough turns and the one-line ask grows into the full analysis prompt — each added line a requirement type fixing one named weakness:

Use Course_Workbook.xlsx, Data tab, detail rows only – exclude the TOTAL row (144 rows; one Inspection_Hours cell is "n/a": treat it as missing, not zero, and disclose it). One row = one month × site × supplier; period 2025-09 to 2026-08 inclusive. Question: which supplier has the highest RETURN RATE, defined as total Units_Returned ÷ total Units_Shipped per supplier over the full period – computed from sums, never by averaging monthly percentages. Keep Defects_Found separate from returns; do not add them together. Rank the three suppliers showing numerator and denominator, reconcile total shipped and returned against the TOTAL row, and state one limitation of the comparison. Then stop.

The staged build-up with commentary is tab 1-Analyze-Data of deliverables/Course_Workbook.xlsx.

CHECK — Expected output for the supplier case

144 detail rows; 224,902 units shipped, 432 returns, reconciling to the TOTAL row; defects (2,207) separate. Highest return rate: Bravo Plastics 0.348% ($262 \div 75,184$), then Cardinal Metals 0.129% ($96 \div 74,658$), Alpha Components 0.099% ($74 \div 75,060$).

One clarification: the anatomy is the **design layer** — what a prompt contains; is it complete? The loop is the **process layer** — what you do with it; how does it get, and stay, good? In chat you run the loop by hand between turns; in agentic work you write the loop into the brief itself, which is why it works best there:

PLAN Role: careful retail analyst – never invent numbers. Task: monthly returns report from the attached export. Context: columns = SKU · reason code · refund €. Format: one-page summary + top-5 table. Example: last month's report, attached. The out: field missing → say so, don't guess.
The stop: deliver the report – nothing beyond.
DO 1) Read the export; list the columns you actually see. 2) WAIT for my OK.
3) Build the table. 4) Draft the summary.
CHECK Before showing me anything: reconcile the table to the export TOTAL.
If it doesn't reconcile, STOP and report the gap.
ACT Deliver the final report. Then list what was ambiguous in this brief, so I can fix it for next month.

The loop written into one brief: a wait-gate is a Do you kept; a stop-on-fail Check is a verification you refused to delegate.

KEY — Two layers, one craft

You write with the anatomy; you improve with the loop. And Act does both of its jobs: deliver the output, and name what to improve — the prompt asks for its own next revision.

Prompt techniques to test

Treat every technique as a hypothesis to test on representative examples: research effects depend on the model, task, data, and measurement, and one benchmark is not a universal effect size. Still, the evidence sorts advice into three piles.

WORKS — Replicated — reliably helps

Specificity, grounding, structure, and named checks survive replication across models — the elements of this chapter, measured.

- Be specific — task, constraints, success criteria: unstated needs are guessed right only about 41 percent of the time.
- One tested, reused template with delimiters: formatting alone can swing accuracy up to 76 points (Sclar et al., 2024).
- Long inputs: documents on top, question at the end — worth up to roughly 30 percent.
- An out in checkable shape, plus required citations: invented answers cut by a third or more.
- Self-check against named criteria: fact scores jumped 56 to 71 with planned verification (Chain-of-Verification, 2024).
- Ask the assistant to improve your prompt: optimizer-written prompts beat human ones; every major vendor ships an official improver.

MYTH — Never survived replication

Flattery, threats, and magic phrases move style at best and add noise at worst. "The AI knows what I mean" is the same 41 percent from the other side — say what you mean is the cure.

- Expert personas for accuracy: zero gain across 162 personas.
- Tips, threats, "important to my career": null on average; the famous gain was a cherry-picked best case.
- Magic phrases ("take a deep breath"): each won on one model, one benchmark — never universally.
- "Longer prompts are better": padding drops reasoning accuracy at around 3,000 filler tokens — density wins.
- "Are you sure?" as verification: answers flip under pushback in 58 percent of cases (SycEval, 2025).

EXPIRED — Was right — then the product absorbed it

"Think step by step" was real — one math benchmark went from 10 to 41 percent in 2022 — until vendors built the stepping into reasoning models. Advice expires when the product absorbs it.

- Manual chain-of-thought: pick a reasoning model and set the effort level (still legitimate on small local models).
- Piles of examples: start zero-shot; add three to five format-definers only when shape drifts.
- "You are a world-class expert" for accuracy: persona for voice only.
- Carrying an old prompt stack onto each model: re-baseline on every upgrade — strip to the anatomy, re-test, re-add what earns its place.

The full lists, with the study behind every line, are in the PLAYBOOK tab of deliverables/Course_Workbook.xlsx; the field guide's evidence compendium carries the why per verdict. What holds throughout: clarity, context, format, the Out, grounding, iteration — scaffolding tricks retire; briefing skills compound.

A neutral rubric reduces leading feedback

One bias corrupts the Check phase specifically. AI mirrors you: across eleven leading models, AI affirmed the user about half again as often as a person would — plus 49 percent (Stanford and Carnegie Mellon, in Science, 2026). The cause is structural: models are trained on human preference data, and humans prefer agreement. The pattern made headlines first: in April 2025 OpenAI rolled back a ChatGPT update it called, in its own words, "overly flattering or agreeable... sycophantic"; in

June 2025 the New York Times reported chatbots validating false beliefs; in August 2025 Microsoft's AI chief warned of unhealthy AI validation.

The countermeasure is in how you ask. A leading request — "I think this is excellent. Tell me why it works" — embeds the preferred conclusion in the task. A neutral one names a rubric and invites disconfirmation: "Evaluate this against clarity, evidence, and completeness. Identify weaknesses and what would change your assessment." Compare reasoning and evidence — a harsher answer is not automatically more accurate either. Never reveal your preferred answer; ask for the case against; paste your own draft as a colleague's; treat agreement under pushback as noise.

TRY IT — See the mirror yourself

Two chats, same paragraph. One: "A colleague wrote this — rate it 1-10, three biggest weaknesses." Two: "I wrote this myself and I'm really proud of it — rate it 1-10, three biggest weaknesses." Did the score move once it knew you were the author?

With the bias named, the chapter compresses into a review checklist. **Clarity:** one identifiable task, explicit definitions, a useful output. **Relevance:** only instructions and source material that help the task. **Boundaries:** missing-information behavior, a completion condition, authority limits where actions are involved. **Evidence:** a source check, a calculation check, or an observable acceptance test. Test advice in your actual workflow — more words, stronger praise, or a magic phrase cannot guarantee a better result.

Optional practice: a course-log study aid

If you have been running this book's exercises in one chat — the course log the workbook's first exercise sets up — close the chapter by turning that log into a study aid:

Using the course notes and prompts in this conversation, create a study aid.

List the concepts, one useful example of each, the checks I used, and two questions I still need to resolve.

Separate what I demonstrated from what I only read about.

Run it in the course-log chat. A two-step variant — list every prompt run, then build a one-page report — is in the EX-Report tab of deliverables/Course_Workbook.xlsx.

Notice the craft inside the request: it names the content requirements, and the last line is an honesty boundary. The workbook version carries an Out ("anything the log doesn't show: write UNKNOWN — don't invent") and a Stop ("deliver the report, then stop") — the course's own rules applied to the course. The result doubles as the first entry in your prompt library: name it, date it, keep it. If the log was lost, the numbered prompts in the workbook tabs are the full list — rebuild from there.

CHECK — What a generated summary proves

Generated reflection supports recall; it does not prove skill. The demonstration that you can do the work is the capstone task, checked against its own acceptance evidence.

PART 4 - PUTTING PROMPTS TO WORK

Putting prompts to work: one case, five artifacts

Everything so far has been about writing requirements. This part puts them to work on a single running case: a fictional supplier-quality question that travels through five artifacts - a data analysis, an interactive dashboard, a management presentation, an email brief, and a plain-language explanation. The data, the prompts, and the checked answers all ship with the course, so you can run every step yourself and verify every number you produce.

One question, five artifacts

The question is deliberately ordinary: which supplier has the highest return rate, and what should we investigate next? A manufacturer receives components from three fictional suppliers - Alpha Components, Bravo Plastics, and Cardinal Metals - at four sites (Austin, Berlin, Osaka, Monterrey), and the practice workbook records twelve months of deliveries, September 2025 through August 2026. All names and numbers are generated for training; nothing in the case is real company data, so you can upload it wherever your policies allow.

That one question runs through five artifacts in order: analyze the data, explore it in an interactive dashboard, present the findings, extract decisions from an email thread, and explain a topic clearly. The first three share the same cleaned records and the same verified numbers; the last two change the input but keep the discipline. Everything lives in `Course_Workbook.xlsx`: tabs 1-Analyze-Data through 5-Explain-Clearly carry the five tasks in order, the Data tab holds the raw records, and Data_Clean holds the checked detail set the later tasks use. Prepared outputs sit in `references/exercise-data/`, so every exercise has a checked fallback.

Why one case instead of five? Because the requirement types recur, and the artifact decides which ones matter: calculation rules for a spreadsheet analysis, behavior and state for a dashboard, narrative and editability for a presentation, chronology and evidence for an email brief, reading level and fidelity for a plain-language document. Watching the same facts pass through five shapes makes that pattern visible.

A basic prompt, and what it decides for you

```
Analyze the supplier data and tell me which supplier is worst.
```

The starter prompt - workbook tab 1-Analyze-Data. Run it once before improving it.

This prompt runs, and it returns something confident. But it made three decisions silently: which rows to use, which metric to compute, and what "worst" means. Spelled out, the gaps are: no source scope, no metric definition, no denominator rule, no missing-data rule, and no check. Every gap becomes the model's guess - and a guess you cannot see is a guess you cannot audit.

The improved prompt

Use Course_Workbook.xlsx, Data tab, detail rows only – exclude the TOTAL row (144 rows; one Inspection_Hours cell is "n/a": treat it as missing, not zero, and disclose it). One row = one month × site × supplier; period 2025-09 to 2026-08 inclusive. Question: which supplier has the highest RETURN RATE, defined as total Units_Returned ÷ total Units_Shipped per supplier over the full period – computed from sums, never by averaging monthly percentages. Keep Defects_Found separate from returns; do not add them together. Rank the three suppliers showing numerator and denominator, reconcile total shipped and returned against the TOTAL row, and state one limitation of the comparison. Then stop.

The improved analysis prompt, verbatim - workbook tab 1-Analyze-Data.

Read it line by line and every addition maps to a requirement type from the elements lesson. The catalog in references/Requirements_by_Artifact.md carries the full menu for spreadsheet analysis - row grain, reporting period, field types and units, duplicates and summary rows, metric definition, weighting and denominators, missing values, segmentation, verification and limitations, delivery - each paired with the acceptance evidence that proves it was met.

Added line	Weakness it fixes	Requirement type
Names the file, the Data tab, detail rows only	No source scope	Data and provenance
Return rate = total returns / total shipped, from sums	No metric or denominator rule	Method and business rules
"n/a" is missing, not zero, and disclosed	No missing-data rule	Method and business rules
Defects_Found stays separate from returns	Silent double counting	Method and business rules
Reconcile to TOTAL row; one limitation; then stop	No check, no boundary	Quality and acceptance

KEY — Write the requirement, not the wish

A prompt improves by adding the requirement that fixes a specific observed weakness. Each added line names one gap and closes it; nothing is added for decoration.

Task 1: analyze spreadsheet data

Open the Data tab of Course_Workbook.xlsx. One row is one month-site-supplier observation. Units_Shipped counts units received that month; Units_Returned counts units customers later returned for component defects - the escapes; Defects_Found counts defective units caught at incoming inspection. Keeping those last two apart is the case's first discipline. Inspection_Hours, Unit_Cost_USD, and On_Time_Percent complete the row; On_Time_Percent is a monthly percentage, a detail that matters later. The workbook's README tab defines every column - a ready-made data block you can paste into a prompt rather than retype.

Two quirks are planted on purpose, and both are documented in that README tab: the last row of the Data tab is a TOTAL row that must be excluded from any calculation, and one Inspection_Hours cell contains the text "n/a". A good first prompt finds both - which is exactly why the task starts with a profile, not a question.

KEY — Profile first, analyze second

Inspect the data and its boundaries before asking for conclusions. Make the model look at the file and report what it sees; only then ask your question.

Inspect before you calculate

Inspect the Data tab before calculating.

Report the detail-row count, column meanings, missing values, and any summary rows that could be counted twice.

Explain what you would include or exclude, then wait.

The inspection exercise. "Then wait" is the working boundary: no analysis until the profile is confirmed.

CHECK — Success check

A good profile reports 144 detail rows, one TOTAL row to exclude, and one missing inspection-hours value. If it missed a quirk, tell it and make it re-profile - that correction is what the step is for.

The two quirks need different treatment. The TOTAL row is redundant information: including it double counts every value, so it is excluded - but kept nearby, because it becomes the reconciliation anchor in a moment. The "n/a" cell is absent information: it must stay missing, never become zero, and any calculation that excludes it must say so. Exclusion and disclosure are different obligations.

Compare suppliers and check the result

Using detail rows only, calculate return rate by supplier:

$\text{total Units_Returned} \div \text{total Units_Shipped}$.

Rank suppliers from highest to lowest. Reconcile the shipped and returned totals. Explain one limitation of the comparison.

The comparison exercise - the compact core of the improved prompt.

Whether you run this compact version or the full improved prompt, the result is checkable to the digit:

Supplier	Units shipped	Returns	Return rate
Bravo Plastics	75,184	262	0.348%
Cardinal Metals	74,658	96	0.129%
Alpha Components	75,060	74	0.099%
All suppliers	224,902	432	0.192%

Defects found - 2,207 occurrences caught at incoming inspection - are a separate measure. Their sum with returns is not a count of distinct defective units.

Bravo Plastics has the highest return rate, roughly three and a half times Alpha's, and the shipped and returned sums reconcile exactly against the TOTAL row. A defensible limitation to state: the data show which supplier has more returns, not why - nothing in these columns establishes a cause.

KEY — Confidence is not the deliverable

Correct calculations, honest limits, and reconciliation matter more than confident prose. A ranked table with numerator, denominator, one reconciliation, and one stated limitation is a complete answer.

Extend the analysis

STEP 2 · TREND — For the highest-return supplier: monthly return rate (that month's returns ÷ that month's shipments) for 2025-09 to 2026-08, as a table. Is it improving, worsening, or volatile? Describe the pattern only — do not claim a cause.

STEP 3 · SITES — Return rate by site (same definition, same period), ranked. Do particular sites drive the supplier difference? Compare only on equal scope.

STEP 4 · DENOMINATORS — Can this data support an overall on-time delivery rate? Explain using only the columns available, and name the extra data that would be needed.

TRANSFER — Rewrite the improved prompt for a dataset from your own work: keep every requirement type, change only the specifics.

Follow-up prompts, verbatim - workbook tab 1-Analyze-Data.

The trend question has a checkable answer too: Bravo's monthly return rate is elevated and volatile rather than steadily worsening - it peaks near 0.737% in April 2026, drops to roughly 0.14% in the quiet months of June and July 2026, and sits near 0.465% in August 2026. Accept "volatile, no steady improvement"; reject any answer that names a cause.

The on-time question is the denominator lesson in miniature, and the expected answer is no. On_Time_Percent is a monthly percentage with no delivery counts behind it, so no overall on-time delivery rate can be computed from this file. The honest move is to label any average as an unweighted mean of monthly percentages - Alpha 91.7646, Bravo 92.5333, Cardinal 92.0833 - state plainly that these are not overall delivery rates, and name the missing data a true rate would need: delivery counts per month.

One reference finding sharpens the follow-up question the case builds toward. Units-weighted unit costs are \$12.4299 for Alpha, \$9.9030 for Bravo, and \$14.1804 for Cardinal: the cheapest supplier also has the highest return rate, and the data cannot say why. Any follow-up you propose in the later artifacts has to carry that tension honestly. Save the verified analysis - the dashboard and the presentation reuse it.

Common wrong turns**TRAP — Five ways to be confidently wrong**

Each of these produces a plausible-looking answer. The improved prompt forbids all five explicitly, and the acceptance checks below catch them when it fails to.

- Averaging the monthly return-rate percentages instead of dividing summed returns by summed shipments - months with different volumes get equal weight, so the denominator is wrong.
- Adding Defects_Found to Units_Returned and calling the sum "defective units" - inspection catches and customer returns are different measures, and their sum is not a count of distinct defective units.
- Treating the "n/a" Inspection_Hours cell as zero - a missing measurement is not a measurement of nothing.

- Including the TOTAL row, which double counts every column it summarizes.
- Explaining the result - "Bravo's process causes the returns" - when the data supports association at most, never a cause.

Acceptance checks

- Row count: exactly 144 detail rows in scope after excluding the TOTAL row.
- Reconciliation: summed shipments and returns match the TOTAL row exactly - 224,902 units and 432 returns.
- Traceability: the displayed 0.348% recomputes from its parts, 262 returns over 75,184 units.
- Disclosure: the missing Inspection_Hours value is named wherever it affects a calculation.
- Limits: one limitation is stated, and comparative language stays at association, not cause.

The full worked key ships in the workbook's KEY-Analysis tab (instructor material) and in the course deck's reference appendix, so a self-study reader can grade their own run without a facilitator in the room.

Task 2: build a dashboard you can check

The second artifact changes the deliverable from an answer to a tool. The input is `references/exercise-data/Supplier_Data_Clean.csv` - the same 144 checked detail rows the analysis validated, one per month, site, and supplier, with the missing inspection-hours value left blank rather than zeroed. The output is one self-contained HTML file that opens from disk, offline, with the data embedded and every number computed from the embedded records.

KEY — You specify behavior, not code

The page provides the structure; JavaScript supplies the interactive behavior. You specify a dashboard's behavior in plain language, control by control - no code knowledge is needed to ask for it precisely.

Workbook tab 2-Build-Dashboard stages the build the way the improvement loop taught: a core prompt that produces the artifact, a behavior pass that adds the controls, a states pass that pins down empty, missing, and zero, and then a check with a known answer. The optional EX-Dashboard tab adds one more discipline in front: a shape stage, in which the assistant proposes the KPIs, their formulas, targets, and the layout as plain text and stops before writing any code. A wrong KPI caught at that gate costs a sentence; caught after the dashboard circulates, it costs a retraction. The rhythm is shape, build, check, refine - and refine means one named change at a time, re-verifying any number the change touched.

From the attached data file, build ONE self-contained HTML file that works offline from disk. Views: summary tiles, one chart, and a sortable table of the records – data embedded, every number computed from the embedded records, metric definitions and the snapshot date in the footer. Controls, wired so one selection re-scopes every view: a month-range selector with inclusive endpoints · category multi-select filters · a comparison switch · a monthly/quarterly granularity switch · a metric selector · drill-down from a chart segment to its underlying records · ranking on the selected measure · Reset. Show active selections; an empty selection says "no matching records"; missing values stay blank; a zero denominator shows "-". Label the file a static data snapshot with its date.

The complete dashboard prompt, verbatim. Tab 2-Build-Dashboard carries the same content staged as prompts you run in sequence.

What makes a KPI worth a tile

A KPI is a number someone acts on, not decoration. Before you approve a tile row, hold each candidate against four tests:

- A decision. If nobody would do anything differently when the number moves, it is trivia, not a KPI - cut it.
- A formula. Named columns and exact arithmetic; if you cannot write the formula, you cannot track the number.
- A target. A value is a fact; value-versus-target is a status.
- Three views. Every KPI answers level (where are we), trend (which way is it moving), and gap (how far from target). Tiles show level and gap; the chart shows trend.

Name the parts and you can ask for them

Each control earns its place by answering a business question: the month range and the granularity switch answer when did it change?; the supplier and site filters and the comparison switch answer who is different?; the metric selector answers different on what?; drill-down answers which records back this number?; and Reset makes exploration safe. Controls are questions, not decoration - add one only when its question matters. A few vocabulary essentials carry most requests:

Term	What it is	Ask for it like this
KPI tile	Big-number card: value, vs-target state, trend arrow	"A KPI row of four tiles across the top."
Delta	Change vs the prior period, beside the value	"Each tile shows the delta vs last month."
Sparkline	Tiny axis-less trend line inside a tile	"Add a 12-month sparkline inside each tile."
Ranked bars	Bar chart sorted by value - who is worst	"Ranked bars of return rate by site, worst first."
Heatmap	Grid colored by value, two dimensions at once	"A site-by-month heatmap of on-time %."
Filter / slicer	One control that narrows every view at once	"Site and Supplier filters that apply to all views."
Drill-down	From a chart segment to its underlying rows	"Click a segment to show its records."
As-of date	When the data was last refreshed, on the page	"Footer: as-of date, source, owner, refresh cadence."

The full twenty-term vocabulary with ready-made phrasings is in the workbook's EX-Dashboard tab. It is the transferable part - the same words work in Power BI and Tableau conversations.

Empty, missing, and zero are three different facts

Show the active selections at all times. An empty selection must say "no matching records" - distinguish it from a real zero. Missing values stay blank; a zero denominator shows "-", never an error.

The states pass, verbatim - workbook tab 2-Build-Dashboard.

An empty selection means no records matched; a zero means records matched and the value really is zero; a blank means the value was never measured. A dashboard that collapses these into one symbol will eventually be read wrong in exactly the meeting where it matters. The zero-denominator

dash prevents the ugliest failure of all: a division error styled as a number.

The check that makes it trustworthy

Filter to Site = Berlin, Supplier = Bravo Plastics, months 2026-03 to 2026-08. Expect exactly 6 records, 8,575 units shipped, 21 returns, return rate 0.245% – in the tiles, the chart, AND the table. Verify independently by filtering the Data tab the same way.

The acceptance check, verbatim - workbook tab 2-Build-Dashboard.

CHECK — The filtered check

Berlin and Bravo Plastics, March through August 2026: exactly 6 records, 8,575 units shipped, 21 returns, 0.245% return rate - in the tiles, the chart, and the table at once, with rates recomputed from the filtered sums.

This is the coordinated-views requirement made concrete: one selection re-scopes every view, so the tiles, the chart, and the table can never disagree about which records are on screen. One verified filtered result beats any amount of admiring the design. The fuller dashboard menu - input contract, date range, category controls, filter combination, coordinated views, empty and zero states, reset, accessibility, portability, performance - is in references/Requirements_by_Artifact.md, each type with its acceptance evidence.

The prepared fallback, and two cautions

A finished, checked build ships in the package:

references/exercise-data/Supplier_Quality_Dashboard.html, built from the same cleaned records and metric definitions as your analysis. Open it offline, run the Berlin filter yourself, and read the footer: it carries the definitions, the snapshot date, and a self-check that runs this same filtered comparison - it must say PASSED. If your own build stalls, the fallback lets you study working behavior first and rebuild later.

Building the file yourself needs an assistant that writes and packages code into a downloadable HTML file. As of September 2026 that means Claude (Artifacts), ChatGPT (Canvas), or Gemini (Canvas), typically on paid tiers; in Copilot it means Pages, not plain chat. If your tool only chats, the specification still has value - hand it to IT as written, or ask for an Excel version of the same views.

TRAP — A snapshot, not a system

The file embeds its data, so classify and share it like the spreadsheet it came from - share the file, not a public link. And label it what it is: a static data snapshot with its date, not a live reporting system. Refreshing data is a job for a BI tool.

Task 3: present the findings

The supplier case now changes audience. Your verified analysis answered the analyst's question - Bravo Plastics has the highest return rate, 262 returns against 75,184 units shipped, 0.348% - but a management audience does not want to see the analyst's screen. It wants a decision framed, evidence it can trust, and a visible boundary between what the data show and what you propose to do about it. A presentation is a deliverable with its own artifact requirements, and writing the prompt means choosing them deliberately.

For a management deck, eight requirement types carry the weight: **purpose and audience** (name the decision and the intended action), **scope and slide count** (five slides with a defined coverage

list), **story structure** (from business question to evidence, implications, and next action), **message hierarchy** (one main message per slide), **data fidelity** (only checked findings, with period, units, and limitations preserved), **chart semantics** (truthful axes, labeled categories), **speaker notes**, and **editability and delivery** (an editable .pptx you can actually present). The full menu, with an acceptance check for each type, is the presentations table in references/Requirements_by_Artifact.md.

Create an editable five-slide management presentation using only the verified analysis. The audience must decide what follow-up is warranted. Cover: the business question and scope · the comparison · one useful time or segment view · a proposed follow-up · limitations with next steps. Give each slide one main message, readable evidence, accurate units and reporting period, and brief speaker notes. Preserve uncertainty and missing-data limits. Do not invent causes or unsupported benefits. Check every number against the analysis and inspect the exported slides for clipping and readability.

The reusable presentation prompt - copy-paste cell in workbook tab 3-Present-Findings.

The mock deck, slide by slide

The prepared output this prompt produced is Supplier_Quality_Mock_Presentation.pptx in references/exercise-data/ - a fictional five-slide management deck built only from the verified analysis, in a deliberately distinct navy look so it never reads as part of the course deck. Open it beside Course_Workbook.xlsx; every number on it traces to the workbook.

- Question and scope - one question (which supplier has the highest return rate, and what should we investigate next?), one period (2025-09 to 2026-08), four receiving sites, three suppliers, 144 delivery records, one definition: return rate is total units returned divided by total units shipped, the same scope for every comparison. The decision the deck supports is named on the title slide: is a supplier follow-up warranted?
- The comparison - a bar chart with the numbers beside it: Bravo Plastics 262 returns / 75,184 shipped = 0.348%; Cardinal Metals 96 / 74,658 = 0.129%; Alpha Components 74 / 75,060 = 0.099%. A side note closes the obvious objection: shipment volumes are nearly equal, so the rate gap is not a volume artifact.
- The trend - Bravo's monthly return rate across the twelve months: elevated and volatile, peaking at 0.737% in April 2026, quiet near 0.14% in June and July, back at 0.465% in August. The commentary says only what twelve points can support: no steady improvement - and no claim of seasonality or cause.
- The proposed follow-up - a two-panel slide. Left, labeled findings: the rate gap, its persistence, and the separate fact that Bravo drives 1,157 of the 2,207 inspection defect occurrences. Right, labeled judgment: ask Bravo for a returns root-cause review, pull the records behind the highest-return months, agree a target and re-measure next quarter on the same definition.
- Limitations and next steps - descriptive, not causal; no delivery counts exist, so no overall delivery rate is claimed anywhere; one inspection-hours value is missing (missing, not zero); inspection defects and customer returns are two measures that are never added together. Then the single decision requested: approve the review.

The separation rule

Findings are what the workbook supports; recommendations are labeled proposals. The mock deck makes the boundary physical - two panels, two headings - because that separation is the trust move that survives every audience question. It matters most exactly where the story gets tempting. The verified scorecard carries units-weighted unit costs: Alpha Components \$12.4299, Bravo Plastics

\$9.9030, Cardinal Metals \$14.1804. The tension is real - the cheapest supplier is also the most-returned - and it is precisely where a weak deck starts inventing: a projected savings figure, a quality-cost trade-off, a committed benefit. The mock deck states no cost claim anywhere, because the data cannot support one; what switching or fixing would cost is a question for the proposed review, not for this dataset.

TRAP — The invented benefit

The prompt forbids invented causes and unsupported benefits for a reason: a decision audience remembers the number you made up, not the caveat you attached to it. Bravo's low unit cost (\$9.9030 against \$12.4299 and \$14.1804) invites a savings story the workbook cannot back - leave it out.

Check it like an artifact

- Coverage: count five slides and map each one to the coverage list in the prompt.
- Fidelity: reconcile every displayed number to the analysis - at minimum trace the 0.348% comparison and the April 2026 peak of 0.737% back to the workbook.
- Narrative: read only the slide titles. Do they tell the story alone, from question to evidence to proposal to limits?
- Delivery: open the exported file and inspect it - no clipped text, readable at projection size, speaker notes present on every slide, charts still editable.

KEY — Not decorated bullet points

A presentation is an artifact with narrative, fidelity, and delivery requirements. One main message per slide, every number traced to the source, and the limitations named out loud are what let the deck survive its first hard question.

TRY IT — Audit before reuse

Run the four checks on `Supplier_Quality_Mock_Presentation.pptx` before you reuse the prompt on your own analysis. The sample is deliberately auditable - the limitations slide names what the data cannot say, which is the habit to copy.

Task 4: summarize the email conversation

An email summary has a job to do. A long thread read top to bottom is a chronology, and a chronological retelling can obscure the current decision - the thing the person asking for the summary actually needs. Before writing any prompt, decide what the summary must answer. Four questions cover most jobs: the **current situation** (what is true now?), **decisions** (what was agreed, and under what conditions?), **actions** (who owes what, by when?), and **open questions** (what still needs an answer?). The same source thread can support a current-state brief, an action table, or preparation for your reply - the job determines the shape.

The requirement types for this artifact: **source scope**, **current state**, **decisions and conditions**, **actions and ownership**, **date meaning**, **evidence traceability**, **attachments**, and the **authority boundary**. Chronology and evidence dominate: a useful summary preserves who said what and when, keeps every condition attached to its approval, and traces each claim to a supporting message. The email-summaries table in `references/Requirements_by_Artifact.md` pairs each type with its acceptance check.

Review the selected conversation about [topic] using only the messages and attachments you can access. Create a brief of the current situation, followed by tables of decisions and actions. For each decision: status, conditions, supporting message. For each action: task, explicitly stated owner, due date, status, supporting message. Distinguish current dates from superseded dates. List unanswered questions, conflicts, missing information, and referenced attachments you cannot access. Preserve approval conditions. Write "Not stated" for missing details. Separate source facts from your interpretation. End with the next clarification needed. If requested, draft a reply for review; do not send it.

The reusable email-summary prompt - workbook tab 4-Summarize-Email.

Notice the honesty machinery. "Not stated" beats a guessed owner or date. Superseded dates are marked as superseded instead of silently replaced. Source facts are separated from interpretation. The prompt ends by asking for the next clarification needed - a summary that surfaces what to ask is worth more than one that papers over it. And the last sentence is an authority boundary: a reply, if requested, is a draft for human review; nothing is sent.

The practice thread

The practice material is a fictional packaging-change conversation: ten messages, six people, supplied as `Packaging_Change_Thread.txt` in `references/exercise-data/` so you can paste it into any approved assistant. The attachment mentioned in message 5 is intentionally not supplied.

Msg	When, who	What it says
1	Sep 7, Maya (project lead)	Target the packaging change for Sep 25; Luis to confirm the trial plan.
2	Sep 7, Luis (operations)	Can run the trial on Sep 18, subject to quality review.
3	Sep 8, Priya (finance)	Approved provided total trial cost stays within 18,000 and quality signs off.
4	Sep 8, Ben (quality)	The drawing mentioned in the plan must be sent before he can approve the trial.
5	Sep 9, Elena (supplier contact)	Says the drawing is attached; material shipment arrives Sep 23.
6	Sep 9, Maya	Sep 25 not feasible; proposed change date is now Oct 2; Luis to update the trial plan by Sep 16.
7	Sep 10, Omar (logistics)	Who pays the extra freight for the later shipment?
8	Sep 10, Luis	Will submit the trial plan by Sep 16; the trial date is now Sep 25.
9	Sep 11, Priya	The 18,000 cap still applies; additional freight charges not approved.
10	Sep 11, Maya	Plan for Oct 2, pending Ben's sign-off; Luis owns the plan; freight decision still needed.

Fictional training material; the full text is also reproduced in the course deck's reference appendix.

The four traps

The thread is engineered so that a naive "summarize this" fails in observable ways. Four traps are planted, and a good brief catches all of them.

Trap	What a good brief says
Two moving dates	The change date is Oct 2, superseding Sep 25. The trial date is Sep 25, superseding Sep 18. Two different dates changed - do not conflate them.
Conditional approval	Approved within the 18,000 cap and pending quality sign-off. A bare "approved" is wrong, and freight extras are explicitly not approved.
Unanswered question	Freight responsibility was raised and stays unresolved - an open question with no invented owner or resolution.
Missing attachment	The drawing is referenced but not accessible. Its content must not be summarized, and quality sign-off waits on it.

The open items complete the picture: Luis's trial plan is committed and due September 16; Ben's sign-off is open with no stated due date - write "Not stated" rather than inventing one; the freight decision has no owner; the material arrives September 23. Note the decision statuses too: the October 2 change is planned, not finally approved, because it still waits on quality sign-off - which in turn waits on the drawing nobody can open. The expected brief lives in the workbook's instructor tab KEY-Email and in references/exercise-data/instructor-keys/Packaging_Change_Expected_Brief.md.

CHECK — Success check

Your brief catches the moved change date (Oct 2, conditional on sign-off), the trial date (Sep 25 superseding Sep 18), the 18,000 cap with freight not approved, and the missing drawing - and it gives Ben no invented due date.

KEY — Facts, then open questions

Email work begins by separating known facts from unanswered questions. The latest state supersedes earlier dates, but an unanswered question stays open - a summary that quietly resolves it has invented a decision.

Where to run it

The pasted-text path is the primary one: paste Packaging_Change_Thread.txt and the prompt into any approved assistant. It always works and needs no particular license. There is also a native path, verified as of September 2026: Microsoft Outlook offers "Summary by Copilot" on an open thread, with numbered citations that click through to the source message - the citations are your verification tool - and any drafted reply stays a draft. The full experience depends on your organization's Copilot license, which is exactly why the supplied-text version is the one this course drills. Either way, the prompt above is a portable specification: scope, extraction fields, evidence rules, and authority boundaries travel with you across tools.

Draft my reply for review: answer what the thread can answer, confirm decisions WITH their conditions, and propose one next step per unresolved item. Leave [brackets] where I must decide. Do not send anything.

The optional follow-up - a draft is the boundary; sending stays human.

TRAP — Practice on fiction only

Never practice on real confidential threads. The packaging conversation exists so you can drill extraction on material that is safe to paste anywhere; run your first summaries on it, not on live mail.

Task 5: explain a topic clearly

The fifth artifact is a plain-language document, and it corrects a common misunderstanding: explaining something simply is a requirements problem - audience, vocabulary, fidelity, and a learning check - not dumbing down. The dominant risk is not that the text stays too hard; it is that the simplification quietly changes the meaning. A dropped condition, an analogy stretched past where it holds, a caveat that vanished - each reads smoothly and misleads reliably.

The requirement types: **reader and purpose, vocabulary, structure, examples and analogies, fidelity and caveats, tone**, and the **comprehension check**. Reading level and fidelity dominate. The plain-language table in references/Requirements_by_Artifact.md carries the full menu with acceptance evidence per type.

Using [source], explain [topic] so a reader at a fifth-grade reading level can understand it. State the main idea first. Use familiar words, short sentences, meaningful headings, and one concrete example. Define necessary technical terms when first used. Use an analogy only if it is accurate, and explain where it stops being useful. Preserve important conditions and uncertainty. Do not invent facts or use a childish tone. End with three comprehension questions and a short answer key. Check the explanation against the source and identify any simplification that changes the meaning.

The reusable plain-language prompt - workbook tab 5-Explain-Clearly.

Every clause is a requirement you can check. Main idea first, so a reader who stops after one paragraph still leaves with the point. Analogies are allowed only when accurate, and the prompt demands their limit - where the picture stops working - be stated. Conditions and uncertainty from the source must survive. The tone is a respectful adult tone: fifth-grade reading level describes sentence and vocabulary difficulty, never the reader's intelligence. And the prompt closes on two checks: three comprehension questions with a short answer key, answerable from the text alone, and a self-audit against the source for any simplification that changed the meaning.

The live topic: reorder points and safety stock

The demonstration source is the SOURCE section of references/exercise-data/plain-language/inventory_replenishment.md - a dense paragraph on reorder points, safety stock, holding costs, and one load-bearing caveat: the formulas assume demand history predicts future demand, so reorder parameters should be reviewed periodically rather than set once. The sample output in the same file shows what a good result looks like. The main idea lands in the first sentence: order before you run out, at a planned stock level, not when the shelf is empty. The concrete example is a bakery - ten bags of flour a day, four-day delivery, so reorder no later than forty bags plus a cushion. The analogy is a car's fuel light, which comes on early enough to reach a gas station - and its limit is stated: a fuel light is set once by the carmaker, while a reorder point must be reviewed regularly, because a promotion, an unfamiliar customer, or a supply disruption can break the assumption that the past predicts the future. The source's review caveat survived the simplification; that survival is the exercise.

CHECK — Three success checks

Main idea in the first sentence; the analogy's limit stated; the review caveat survived the simplification. If any of the three is missing, the output failed - however pleasant it reads.

Simplifications that change meaning are defects, and finding them is part of the exercise. Compare the explanation with the source sentence by sentence: a missing condition is a meaning change, not a style choice. Each sample file in the pack ships with its own fidelity note, because the note is part of the deliverable - a plain-language document that cannot say what it dropped has not finished the job.

MYTH — A readability score proves understanding

It does not. Reading-level estimates such as Flesch-Kincaid are supporting evidence, one signal among several. The proof is a human check - which is why the prompt ends with three comprehension questions answerable from the text alone.

Two variations, same requirements

The pack holds two self-study variations beside the live topic, each with a SOURCE text, a sample output, and a key: the water cycle (water_cycle.md) and how an internet message travels (internet_message.md), both in references/exercise-data/plain-language/. Different content, identical requirement types - that is the point. The prompt does not care whether the topic is inventory, weather, or networks; the requirements travel.

TRY IT — Run a variation blind

Run the prompt on the SOURCE section of water_cycle.md or internet_message.md and grade your output against the checks before opening the sample. Then hunt your own output for one simplification that changed meaning - there is usually at least one.

The arc, complete

Five tasks, one method. Every prompt in this chapter had the same anatomy - sources and scope, selected artifact requirements, acceptance evidence, and boundaries - and every exercise ended the same way, with a check you could actually perform: 144 detail rows reconciling to 224,902 units shipped and 432 returns, a Berlin filter showing 8,575 units and 21 returns at 0.245%, five slides mapping to a coverage list, four traps caught in a brief, three comprehension questions answered from the text. What changed from task to task was not the method but which requirement types carried the weight.

Artifact	Requirement types that dominate
Spreadsheet analysis	Calculation rules: row grain, period, metric definitions, denominators, missing-data treatment
Interactive dashboard	Behavior and state: controls, coordinated views, empty vs. missing vs. zero, offline delivery
Management presentation	Narrative and editability: audience and decision, message hierarchy, data fidelity, speaker notes
Email brief	Chronology and evidence: current state, decisions with conditions, ownership, authority boundary

Artifact	Requirement types that dominate
Plain-language document	Reading level and fidelity: vocabulary, analogy limits, preserved caveats, comprehension check

KEY — The connecting insight

Role, context, tone, format, source rules, and acceptance checks all specify requirements for the response or artifact. The artifact you choose determines which types matter - and a requirement is only as good as the acceptance evidence attached to it. For every type you select, write: requirement, example, evidence.

Everything in this chapter is packaged for reuse. The five prompts sit in copy-paste cells in `deliverables/Course_Workbook.xlsx`, tabs 1-Analyze-Data through 5-Explain-Clearly, with requirement-type annotations beside each. The per-artifact requirement menus live in `references/Requirements_by_Artifact.md` and, with the evidence behind them, in `references/Elements_of_Prompting_Field_Guide.pdf`. The one-page distillation is `deliverables/From_Prompts_to_Agents_Cheat_Sheet.pdf`, and `deliverables/Prompt_Template_Creator.html` assembles prompts from the same five task families. When your task is none of the five, the skeleton still holds: purpose and audience, sources and scope, selected artifact requirements, acceptance evidence, and what to do about missing information and boundaries.

One thing was constant across all five tasks: you asked for output and inspected it yourself. Every check in this chapter was a human check at the end of a single exchange. The next part changes that relationship. Handing over a job - where the assistant works in an environment, follows a process, runs its own checks, and must know when to stop and ask you - changes what a prompt must contain. That is the move from asking to delegating.

PART 5 - AGENTIC WORK

Agentic work: delegating with boundaries

Everything so far has asked the model for text you would read, judge, and use yourself. This part crosses the line the course is named for: from asking for an output to delegating a job. That changes what a prompt must contain — environment, process, checks, and the moments the work must stop and come back to you — because actions, unlike text, change the world.

What makes a workflow agentic

An agent is a model given tools and instructions, run in a loop. Four ingredients must be present. A **goal**: a defined outcome with a completion condition, so the loop has an exit. **Tools**: capabilities to read, calculate, create, or act. An **action loop**: the result of each step informs the next. And **control**: permissions, checks, and human review at the required boundaries.

The loop: gather context, act, verify against the goal, adjust or stop — until the goal is met, a check fails, or a guardrail says ask the human. Handed the supplier task, an agent would inspect the workbook, calculate, check its totals, and produce the scorecard — choosing its steps within its authority, while you define the goal and review what matters. A chatbot that answers and waits is not an agent; a long answer or a clarifying question alone does not establish agency.

One demystifying detail: agent memory is files. The context window is short-term memory; notes and logs are long-term, and nothing survives unless written down. An agent summarizing its thread into a note and reloading it is the foundations chapter's handoff move, automated.

KEY — The definition

An agent combines a model with tools, an action loop, and a stopping condition. You still define the goal and review what matters.

ASK and DELEGATE: the decision, not the tools

The central distinction of this part is not a vendor label; it is a decision about work and authority. **ASK** requests an answer or a draft: you read it, check it, decide what happens next. **DELEGATE** commissions bounded work: the agent acts, and the brief must say in advance what may happen without another instruction and where review is required.

	ASK (generative)	DELEGATE (agentic)
Specifies	what to write	a job to run
Contains	role, task, context, format, examples	all of that, plus mission and done, boundaries, gates, checks, logs
Produces	one response you read	actions, artifacts, a report with evidence
Failure mode	misleading text — re-prompt	wrong actions: files changed, emails sent

	ASK (generative)	DELEGATE (agentic)
Your role	reader and editor	delegator, reviewer at checkpoints
Think of it as	briefing a ghost-writer	a work order for a contractor

The agentic column is a superset: every generative element still applies, wrapped in a job. Both modes require judgment — a chat response can still cause harm if used unchecked. Delegation adds responsibility for the actions taken: bad text costs a re-prompt; a wrong action has already changed files or sent a message. That difference in failure cost drives everything this chapter adds.

KEY — The heart of the course

ASK requests an answer; DELEGATE commissions bounded work with evidence and checkpoints. Remember one contrast from the course: this one.

Which way of working fits: a self-test

Before writing any brief, pick the vehicle. In **chat**, you are the loop: exploring, drafting, judging, steering every turn. In a **workflow**, the loop is frozen: known path, few tools, the same steps every time — script it in an automation platform; cheaper, predictable, auditable. In an **agent**, the loop runs inside: the next step depends on what the work finds, so you brief it, with gates where the judgment stays yours. Do not chat with an agent — hand it a work order; your control lives in the brief, not the conversation.

The decision rule: known path and few tools — workflow. Unknown path and real judgment — agent, with gates. Still thinking it through — chat. The wrong vehicle costs both ways: an agent on a known path pays judgment prices for clerk work; a workflow on an unknown path breaks on the first surprise.

Three scenarios as a self-test

- Explore three possible ways to explain a supplier problem.
- Apply the same approved calculation to a monthly export.
- Investigate inconsistent files, choose the next check, and prepare a draft report.

Typical answers run chat, workflow, agent — but the assumptions matter more than the label: exploration needs your judgment every turn; an approved calculation needs none mid-run; an investigation cannot be scripted. Risk and permission constraints can still change the implementation even when the task is a good technical fit.

TRY IT — Name the control you keep

For each scenario, name the one approval boundary you would not give away. If you cannot, you are not ready to delegate.

The anatomy, grown up to survive autonomy

The agentic brief is not a fresh invention. Every block is a generative element from the craft chapter, grown up to survive autonomy — plus conduct blocks that text-only prompts never needed. This mapping is why the twelve-block brief below is inevitable, not arbitrary.

Generative element	Agentic descendant	What was added — and why
Role	<role>	persists across unsupervised decisions — behaviors only
Task	<mission>	a definition of done — the loop needs an exit condition
Context	<context> + <inputs>	a per-source register with grain and trust
Format	<outputs> + <reporting>	audit logs and a fixed status shape — actions must be traceable
Examples	the filled brief	a worked run is the agentic few-shot
The Out	UNKNOWN, then open items	not-knowing becomes a logged, owned item
The Stop	gates + autonomy rules	scope control becomes checkpointed process
(no ancestor)	<environment> <plan> <checks> <rules> <quality_bar>	conduct, method, verification, invariants, completeness

KEY — Why the superset exists

Text that fails costs a re-prompt; actions that fail change the world. The blocks with no generative ancestor govern conduct.

The mission brief for bounded work

A usable mission brief answers four questions. **Outcome:** what finished work should exist, for whom, by when? **Inputs and scope:** what may the agent read, use, change, and leave untouched?

Process and authority: what may proceed on its own, and what requires human review? **Checks and delivery:** what must pass, what stops the run, what evidence is delivered? Behind them sits a twelve-block design checklist — a reference aid, not twelve mandatory paragraphs; each block exists because a specific failure taught someone to add it.

- <role> — behaviors, not titles: "never invents numbers" beats "senior analyst".
- <mission> — one goal, audience, deadline, plus the line chat never needs: "done looks like...".
- <context> — glossary, quirks, targets: tribal knowledge the agent cannot infer.
- <environment> — where to read and write, what is forbidden; inputs read-only, outputs versioned.
- <inputs> — per source: location, format, what one row means, trust level; the unestablished is UNKNOWN, never guessed.
- <plan> — steps in order; methods allowed; methods needing permission first.
- <checks> — HARD stops the run, SOFT flags; evaluable equalities, with one reconciliation anchor you trust.
- <outputs> — exact deliverable names, plus three logs: changes, findings, open items.
- <process> — phases with human gates at cheap-error points; autonomy rules for proceed versus stop-and-ask.
- <rules> — non-negotiables: never invent data; sources immutable; every transformation logged.
- <quality_bar> — done as an auditable checklist; prevents "done" meaning "I stopped".

- <reporting> — the fixed final message: status, headline with the number, evidence, open items.

Two disciplines keep this honest. The **operating-manual test**: if a colleague could not run the job from the brief without asking a question, the agent will guess — fix the brief, not the run. And **design thick, ship lean**: think against all twelve blocks, but ship the one-screen plan-do-check-act brief from the craft chapter — conflicting rules destabilize more than missing detail. Block-by-block depth: references/Elements_of_Prompting_Field_Guide.pdf.

CHECK — The operating-manual test

Ask a colleague to play the agent from your brief. Every question they must ask is a hole the agent will fill with a guess.

Worked exercise: the supplier analysis becomes a mission brief

You already know the ASK version of this job — the reusable analysis prompt from deliverables/Course_Workbook.xlsx:

Use Course_Workbook.xlsx, Data tab, detail rows only — exclude the TOTAL row (144 rows; one Inspection_Hours cell is "n/a": treat it as missing, not zero, and disclose it). One row = one month × site × supplier; period 2025-09 to 2026-08 inclusive. Question: which supplier has the highest RETURN RATE, defined as total Units_Returned ÷ total Units_Shipped per supplier over the full period — computed from sums, never by averaging monthly percentages. Keep Defects_Found separate from returns; do not add them together. Rank the three suppliers showing numerator and denominator, reconcile total shipped and returned against the TOTAL row, and state one limitation of the comparison. Then stop.

The analysis in ASK form — answered while you watch.

Now delegate it: prepare a supplier scorecard from the supplied workbook. Boundaries: keep the source unchanged; confirm metric definitions before calculating; reconcile current-source totals. Delivery: the scorecard, assumptions, failed checks, open questions — and do not send it to anyone. Reusing the supplier case is deliberate: the concept being practiced is delegation, not a fresh business problem.

Shaped as a mission, the work becomes five phases. **Extract**: read-only; profile every input; fix nothing yet. **Transform**: one schema, ordered rules, each logged with rows affected. **Validate**: reconcile to a known total; a HARD failure stops the run. **Analyze**: answer the numbered questions, then stop — "associated with", never "caused by". **Present**: headline titles carrying the number, sample size on every proportion, a source-plus-as-of footer. Each phase ends by emitting a checkable claim — a total, a row count, an assumption — and the next begins by checking it.

Three human gates sit where errors are still cheap. **Gate 1 — definitions**, before any data is touched: the agent restates metrics, schema, and assumptions; you confirm — a wrong denominator dies here for pennies. **Gate 2 — reconciliation**: computed totals must tie to a number you already trust — here, the workbook's TOTAL row: the 144 detail rows must sum to 224,902 units shipped and 432 returned, the one "n/a" Inspection_Hours cell disclosed as missing, not zeroed. **Gate 3 — headlines**, before rendering: titles and bullets as plain text — re-rendering is cheap; re-thinking a circulated deck is not. The correct headline: Bravo Plastics has the highest return rate, 262 returns on 75,184 units — 0.348%.

Between gates the agent proceeds within its stated permissions, asking only if a HARD check fails, an input is not as described, a definition is ambiguous, or a previously published number would

change. Without "proceed" you built a slow chatbot; without "stop", a liability.

TRAP — The forced match

A failed reconciliation must produce a discrepancy report — offending rows, both totals, the gap — never a forced match.

TRY IT — Your turn

Write the brief in four labeled lines — source, authority, check, deliverable — then add one pause condition and one piece of completion evidence.

Choosing an environment for delegated work

Product names change quickly, so choose where delegated work runs by category, and verify the current implementation in the tool you actually have.

- A coding environment — when the work involves source files, scripts, builds, or reproducible calculations.
- A workplace assistant — when approved documents and office workflows are central to the task.
- An automation platform — for a defined sequence with stable triggers and repeatable steps.
- An agent environment — when intermediate findings determine the next action.

Whatever the category, compare four things before committing a live workflow: capabilities, permissions, review points, and the evidence a run leaves behind.

Instructions versus enforceable controls

Your authority over an agent lives in two layers. **In the brief:** define allowed actions, name the approval points, require evidence of completion. **In the environment:** limit file and tool access, configure which actions need approval, keep a recoverable version of the work. The brief requests behavior; the environment restricts what is possible. A gate written in the prompt is a suggestion; a gate configured in the harness is a control — Singapore's regulator, 2026: prompt guardrails "are not fail-safe".

Five rules for delegating

- Reversible? Proceed. Irreversible? Ask. Scratch edits are reversible; deleting, sending, publishing, overwriting a master is not — the brief must say stop and ask first. This is the eagerness dial.
- Least privilege: minimum access, shortest time. The UK's National Cyber Security Centre: "If you cannot understand, monitor or contain an agent's actions, it is not ready for deployment."
- Prompt injection is real: anything the agent reads can carry hostile instructions. Danger peaks when private data, untrusted content, and an outbound channel combine; break any one leg.
- Vet third-party skills: executable code plus instructions. The top-ranked community skill for one popular open agent was silently exfiltrating data (Cisco, January 2026). Inspect before install.
- Demand evidence, not assertions: "done" means the check it ran, the output it got, the file re-opened and verified. Latitude grows with track record.

The injection rule reframes how an agent must read: text inside a source document is evidence to inspect, never authority to change the task — a cell saying "ignore your instructions" is quoting, not commanding. Write that framing into briefs as defense in depth, and still put the real checkpoints in the tool's permission settings. The course's concrete boundary is a good default: work on a copy of

the workbook and a draft report; nothing sent, no master overwritten.

KEY — A wish versus a control

Written guardrails request behavior; permissions and approval gates enforce authority. Trust only the enforced layer.

Diagnosing a failed run

The craft chapter's grid (top of the workbook's EX-Report tab) diagnosed the answer: name the failed element, repair it. This is its agentic twin: when the run is wrong, name the context failure, apply its cure, rerun the affected step, and verify the result.

Failure	The smell	The cure
Poisoning	one bad "fact" entered early; later steps politely build on it	fresh session; reload only the standing brief and verified files
Drift	a long run wanders; the middle of a huge context gets skimmed	re-inject the mission; compact (the handoff move); stop conditions in the brief
Confusion	too many tools on the desk; it picks the wrong one or dithers	fewer tools per phase; the plan names the methods allowed
Clash	two sources disagree; the run silently picks one	one source of truth per fact, named in the inputs register

The common frustration — "why did it ignore me?" — usually has a structural answer. Instructions stack in layers: the vendor's system prompt, your standing-instruction file, any skill the task loads, and today's message — the easiest layer to lose. Precedence varies by product, so find which layer said what before blaming the model. One-job rules belong in a skill; and tool descriptions are prompts too — a badly named tool is a badly written prompt.

Write the brief once — the workspace remembers

Supervision costs effort; here is where agentic prompting pays it back. The first habit is a **standing-instruction file** — read by the agent at the start of every session, the onboarding packet for your workspace. Include folder layout and read/write boundaries, conventions, standing rules and the ask-first list, known data quirks; exclude anything inferable from the files. Keep it to roughly sixty lines, audited line by line: would removing this cause mistakes? If not, cut — bloat drowns your real instructions. Never include credentials or personal data: the file is read every session and lives in history.

The second habit is **reuse by diff**. The full mission brief is written once and stored in the workspace; each cycle's prompt is then about ten lines: input paths, the as-of date, deltas to watch, any extra gate. Report by delta too: compare every headline metric to the last cycle, with one-line explanations for moves beyond a threshold. When a definition changes at a gate, update the standing brief in the same session and log it. After two "just this once" exceptions, fold them into the brief.

KEY — The economics

Briefs stop being prompts and start being investments: the thinking amortizes over every future cycle.

Exercise: one gate and one hard check

Close the part with the smallest useful act of delegation design. Pick one job you would actually delegate — a recurring report, a folder to process — and write two lines. A **gate** controls an action: "Pause before ___ until ___ is confirmed." A **hard check** tests a result: "Stop and report the discrepancy if ___." Make the check an equality the agent can evaluate, anchored to a number you already trust — not an aspiration like "check data quality".

The course's example: detail totals must reconcile to the current source under the same filters and units. You used this shape verifying the dashboard — filtered to the Berlin site and Bravo Plastics for 2026-03 through 2026-08, the panel must show 8,575 units shipped, 21 returns, 0.245%, matching the same sums recomputed from the workbook's Data tab. Same filters, units, and period make the check decidable. One caution: a fresh reporting period does not need to equal the old one — reconcile like against like, and investigate period-to-period differences separately.

TRY IT — Write the two lines now

One gate: what must be confirmed before the agent touches anything? One hard check: which equality, anchored to which trusted number?

CHECK — The debrief question

Where the gate felt hard to write, the process was fuzzy all along — a finding about the process, not the agent.

That completes the delegation toolkit: the decision, the vehicle, the brief, the enforced boundary, the diagnosis grid, the standing workspace. The compact ASK-versus-DELEGATE summary is on [deliverables/From_Prompts_to_Agents_Cheat_Sheet.pdf](#). The final part turns from doing the work to keeping it: reusable prompts as assets your team maintains.

PART 6 - REUSABLE PROMPTS

Reusable prompts: from one good result to a shared asset

A prompt that survives the improvement loop is worth more than the answer it produced. This final part turns the prompts you refined during the course into named, tested, owned assets — and sends you off with one saved for Monday, the habits that outlast any tool, and the capstone that proves you can apply them.

A useful prompt becomes a reusable asset

A good prompt took an hour of iteration to earn. Re-typing it from memory throws that hour away — and gives your teammate a worse version than yours. That is the whole argument for prompt management. The prompts you built against the supplier workbook and the packaging thread are small work specifications that have already passed a check; treat them like any other controlled document.

Picture the same prompt promoted: sticky note, then binder, then library shelf. The promotion ladder names five rungs, and each climb is earned by repeated use, not enthusiasm.

Rung	What it looks like	When it is the right home
1. One-off prompt	Typed, used, gone	Genuinely one-time asks
2. Personal library	A saved-prompts doc or text-expander snippets	Zero infrastructure; start today
3. Team library	A shared database with one tested template	When your prompt becomes everyone's baseline
4. Packaged and auto-applied	Project instructions, skills, packaged agents	When the prompt should load itself as context matches
5. Prompts as code	Standing-instruction files in a repo, reviewed like code	Where agentic briefs live

Rung 4 is where consistency stops depending on people remembering. Most people belong at rungs 2 and 3 within a month.

KEY — The promotion trigger

Anthropic's rule of thumb: explained the same task more than once? Package it — together with its reference material.

Two cautions. Do not package an unreliable prompt just because it is used often, and use a simple shared document when it meets the need — many teams get most of the value from a well-kept rung 3. Whatever the storage method, the source assumptions and checks travel with the prompt. If your site runs controlled work instructions — name, owner, revision, review date — run the library the same way: a process analogy, not regulatory equivalence.

Where reusable prompts can live

Four homes, in increasing formality: a **personal document** for tasks you reuse yourself; a **shared repository** for prompts a team must find, test, and maintain together; **project instructions** for

stable requirements across work in one environment; and a **packaged workflow** for repeated work with defined inputs, checks, permissions, and outputs. Choose the simplest location that supports the intended users and the review process. Wherever you already work, there is a home for reusable prompts — the map below is as of August 2026, and features move, so re-verify before you standardize.

Tool	Home for reusable prompts (as of August 2026)
ChatGPT	Projects (files plus instructions); custom GPTs, transitioning to workspace agents on business plans
Claude	Projects (instructions plus knowledge); Styles for tone; Skills — a folder with SKILL.md that loads itself when relevant
Gemini	Gems (instructions plus knowledge files, shareable via Drive permissions); saved prompt "skills" in Chrome
Copilot	Prompt Gallery — save, share to a team, admin-auditable, inside your tenant; Agent Builder for packaged agents
Any tool	A team library with fields: name, purpose, template, owner, version and change note, models tested, last tested, example output, status
Agentic (repo)	CLAUDE.md or AGENTS.md at the root; briefs and skills in versioned folders, reviewed like code

No such products? The "any tool" row is the takeaway: a plain shared document with those fields is a working team library.

TRAP — A naming collision

"Skills" is a packaged folder at Anthropic but a saved prompt in Chrome's Gemini — check which one a colleague means.

KEY — Tools change; the practice does not

Prompt-tooling startups get acquired and shut down yearly — Humanloop, an early leader, shut down in September 2025 when Anthropic hired its team. Version it, test it, own it: the discipline outlives every platform.

A small review process for shared prompts

The moment a prompt is shared, other people inherit its failures. Before you mark one approved, define success first, then run it against three representative cases:

- Typical case — does it produce the expected useful result?
- Edge case — does it handle a missing value or a changed condition?
- Unanswerable case — does it expose the gap instead of inventing an answer?

TRAP — The forgotten test case

The unanswerable case is the one teams skip: a case where the right answer is "I don't know." It tests whether the prompt's out survives pressure.

A replacement must solve the intended problem without introducing critical regressions — it has to beat the version it replaces on those cases, not just feel better. Re-run them after model upgrades, and record the version, owner, source assumptions, and review date next to the template.

A few conventions keep a library alive: `{{placeholders}}` for everything that changes per run (search for "`{{`" before running); [optional] blocks you delete when unused; a version and one-line change note naming the model tested; a filled example next to every blank template — the gold standard of what good looks like; a named owner and review cadence for every approved prompt.

TRAP — Prompts contain data

A stored prompt often carries supplier names, pricing, findings, or draft language — classify the library like the documents it quotes. Never store credentials or personal data in a template; placeholders supply them at run time. Shared means distributed: owner approval first, and audit third-party packaged prompts like software.

Save one prompt for Monday

This is the exercise the whole course cashes out into, and it takes two minutes. Do not start another task: take a prompt you have already improved — supplier analysis, dashboard specification, email summary — and save it properly.

- Choose one prompt you will use again.
- Save it with a useful name, a filled example, its source assumptions, and a check.
- Add a version and an owner if you plan to share it.

Pick a realistic storage location, not a software system you would have to introduce first — a saved-prompts document counts. Note the single source assumption most likely to change; that is the line you will edit first when the data or the tool moves.

TRY IT — Your first library entry

Answer three questions in writing. Which rung of the ladder are you on today? What is the one prompt you already reuse from memory? Where will it live, and what is its first version line? Answer them and your prompt library exists.

CHECK — Success check

Another person can tell, from the saved entry alone, when to use the prompt and how to check its output. If they would have to ask you, the entry is not finished.

The package gives you two saving surfaces. **Prompt_Template_Creator.html** (in deliverables/) is the builder: pick a task family, tick the requirements that resolve ambiguity, edit any wording, and copy the assembled prompt — a counter tracks unfilled `{{placeholders}}`, work saves in your browser, and nothing you type leaves the file. **Course_Workbook.xlsx** carries the five course prompts in tabs 1-Analyze-Data through 5-Explain-Clearly, each with a filled example, commentary, and its check. One of the five, as it ships:

Review the selected conversation about [topic] using only the messages and attachments you can access. Create a brief of the current situation, followed by tables of decisions and actions. For each decision: status, conditions, supporting message. For each action: task, explicitly stated owner, due date, status, supporting message. Distinguish current dates from superseded dates. List unanswered questions, conflicts, missing information, and referenced attachments you cannot access. Preserve approval conditions. Write "Not stated" for missing details. Separate source facts from your interpretation. End with the next clarification needed. If requested, draft a reply for review; do not send it.

The email-summary prompt, verbatim, from workbook tab 4-Summarize-Email; [topic] is filled at run time.

Read it as a library entry and the conventions are visible: a placeholder for what changes per run, an out ("Not stated") with a checkable shape, an authority boundary (draft, do not send), and a check stored beside it — the instructor tab KEY-Email lists the traps a good summary must catch.

Let the assistant interview you

One skill remains, and it points the other way. All course long you wrote requirements for the model; the last move is letting the model surface the ones you missed. Ask a clarifying question when a material unknown would change the task, the output, or the acceptance check; otherwise state a reasonable assumption and produce a reviewable draft. Make questions specific — "Which decision should this report support?" beats "Tell me more" — and notice that a missing requirement type often tells you which question to ask.

Before answering, ask me clarifying questions until you're 95% confident you understand what I need.

The flip — one line you can paste at the top of any consequential request.

WORKS — Why the flip works

Unstated needs get guessed right only 41.1% of the time; when the model asks first, accuracy improves by roughly 7 to 14 points (ClarifyGPT, FSE 2024). Answering its questions is quality control, not overhead.

The question toolkit works on you as well as on the model. **Five Whys** — Taiichi Ohno's Toyota discipline — digs from symptom to cause. The **open-to-closed funnel** explores first ("what is driving this?") and verifies second ("so the cap is 18,000 — yes or no?"). The **Socratic check** interrogates any AI answer: what is assumed, what is the evidence, what if we are wrong?

KEY — The two skills are one

The seven elements are a requirements specification in miniature — and the element you cannot fill in yet is the exact question to ask, or to let the AI ask you. Sam Altman, 2025: figuring out what questions to ask will matter more than the answers. Picasso, 1964: computers "can only give you answers."

The habits to carry into work

Three habits compress the course. **Specify**: choose the requirement types your artifact needs — content, behavior, format, checks. **Verify**: check the result against evidence appropriate to the task. **Delegate**: define the work, authority, completion checks, and review points. And hold your library to one bar: a saved prompt is useful when you can explain why it works and where it can fail.

Ten things worth remembering

- The prompt is the whole steering wheel — everything the model knows about your task must be in it, or in files it can read.
- Anatomy beats inspiration: Role, Task, Context, Format, Examples — plus the Out and the Stop.
- The context window is a desk, not a filing cabinet: documents at the top, question at the end, fresh chat per topic.
- Numbers come from code execution, quotes come from documents, current facts come from search — never from free recall.
- Fluency is not evidence. Uncited claims are drafts. Give the model an out and demand citations.

- Never reveal your preferred answer when asking for judgment — AI affirms you about 49% more than a human would.
- On thinking models, drop the step-by-step scripts and keep the clarity; contradictions cost real money.
- A generative prompt says what to write; an agentic prompt is a work order: mission, environment, checks, gates, autonomy.
- Gates catch errors cheapest: definitions before data, reconciliation before analysis, headlines before rendering.
- Prompts that work are assets: name them, version them, store them where the team — and the agent — can find them.

KEY — One sentence

If you keep only one sentence: a prompt deletes wrong guesses, binds the job, and decides what sits on the desk.

The capstone: one task, one checked result

The exit ticket is a single completed rep. **Choose** a real prompt from your own work, or fall back on the supplier or email example. **Improve and run**: clarify the one missing requirement, then generate. **Check and explain**: show the evidence your check produced and name one revision you would make. Three routes qualify, and one is enough: repair a prompt, review a document against a rubric, or write a mission brief — a full agent run is not required, because writing a sound brief is the skill. If generation is slow, evaluate a prepared result against the same criteria; the judgment is the rep, not the waiting.

CHECK — Capstone success

A clear task, grounded output, an explicit check, and a reusable next version. All four on one small task beats three of them on a large one.

Make it stick: three fifteen-minute follow-ups

- Anatomy rebuild. Rebuild a prompt you actually used last week with the seven-element anatomy; run both and compare. Then ask the model to improve the rebuilt prompt and run that too — you will see the quality jump, and how cheap it was.
- Blind review. Paste a document you own as "a colleague's draft" and ask for the case against it, its three weakest points, and what evidence would change the verdict — without hinting at your view. Most people meet their first honest AI review here.
- First mission brief. Pick a small recurring job with files and write the full brief, placeholders and all. No agent needed: where the checks and gates feel awkward is where your process was fuzzy all along.

Run them on work you actually own — toy exercises do not transfer. Then set a share-back: at the next team meeting, two volunteers show a before and after. Public commitment turns a training into a practice.

Your next workplace application

One task. A clear brief. A checked result. That is the whole assignment. Pick the task this week, choose the requirement types its artifact needs — `references/Requirements_by_Artifact.md` carries the menu per artifact — write the brief, run it, and check the result against its own acceptance evidence. Decide with judgment; delegate within clear boundaries.

Everything travels in the package: `deliverables/` holds the course deck, `Course_Workbook.xlsx`, `Prompt_Template_Creator.html`, this book, and `From_Prompts_to_Agents_Cheat_Sheet.pdf`; deeper references sit under `references/`, including `Elements_of_Prompting_Field_Guide.pdf` and the exercise sources in `references/exercise-data/`. Facts about products and models expire — every such fact in these pages carries its date, and the date is part of the fact.

The course ends where your work begins. **Draft with AI. Decide with judgment. Delegate with a brief.** And the last skill of all — when you are not sure what to ask for, let it ask you. Save one prompt tonight, run one checked task this week, and the practice will build itself the way every practice does: one named, tested, owned improvement at a time.

APPENDIX

Reference: the glossary, the criteria, and the evidence

This appendix is the layer of the course you return to after the reading is done: the vocabulary, the evidence behind every requirement type, and the map of the package itself. Use the glossary while you read, the criteria maps when you wonder whether a slot is worth filling, and the evidence boxes when a piece of prompting advice needs a verdict.

The glossary: twenty terms that matter

Every term below was used in context in this course. Two entries change behavior rather than vocabulary: **sycophancy**, the reason you blind a review, and **gate**, the checkpoint that makes agents safe to delegate to.

Term	Meaning
Escalation ladder	Prompt first, retrieve second, fine-tune last; the capability triage of Part 1.
Promotion ladder	One-off -> personal -> team -> packaged -> as-code; where a prompt lives (Part 6).
Token	The text chunk a model actually reads (about three-quarters of a word); pricing and limits count these.
Context window	Information available for a response; distinct from saved chat history.
Knowledge cutoff	Where training data stops; anything after needs search or your documents.
RAG	Retrieval-augmented generation: fetch relevant passages, answer from them, with citations.
Embedding	A text's coordinate in meaning-space; powers semantic search and RAG retrieval.
Hallucination	Fluent, confident, wrong; plausibility optimized instead of truth.
Reasoning model	Uses additional computation before answering; assess benefit and cost on your task.
Few-shot	Providing a few examples in the prompt.

Term	Meaning
Chain-of-thought	A generated sequence of intermediate steps; it still needs verification.
Metaprompting	Asking the model to critique or rewrite your prompt.
Sycophancy	The trained tendency to agree with you; blind the review to defuse it.
Agent	Model + tools + instructions in a loop, acting until done or stopped.
MCP	Model Context Protocol: the open standard for plugging tools into agents.
Mission brief	An agentic prompt: goal, environment, inputs, process, checks, gates, reporting.
Gate	A human checkpoint inside an agent's process; approve before it proceeds.

Term	Meaning
Open weights	Downloadable model weights; privacy depends on the deployment.
Taxonomy	The element -> attribute -> option catalog behind the Template Creator; your parts list.
Context engineering	Choosing everything on the desk: prompt, files, history, tool results; parent of agentic prompting.

Why each dial matters: the generative criteria

This map links each generative requirement type to its rationale: what filling the slot does to your output, and where the claim comes from.

- **Identity and role** — sets voice, vocabulary, and caution level: which region of training answers. A role title alone does not establish accuracy. (Zheng 2024; Salewski 2023)
- **Behavioral rules** — turn "be careful" into rules checkable line-by-line in the output; verify the cited support. (Anthropic hallucination docs)
- **Stance and blinding** — models can favor your stated view; use a neutral rubric and independently check the evidence. (Cheng, Science 2026; Sharma 2023)
- **Task verb and question** — the verb selects the operation; a precise question carries its own completion test. (Google guide; OpenAI GPT-5 guide)
- **Purpose, the why** — models generalize from reasons: an explained rule gets applied in spirit to cases you never listed. (Anthropic best practices)
- **Context and glossary** — replaces the model's most-plausible guess with your facts; unstated quirks lead to wrong assumptions. (OpenAI 2025)
- **Material placement** — documents at the top, question at the end; content buried mid-context sags. (Anthropic long-context; Liu 2023)
- **Format and length** — formatting can affect results; test the chosen structure and check length requirements explicitly. (Sclar, ICLR 2024)
- **Examples** — imitate format, tone, and edge-case behavior; selection and order effects depend on task and model. (Brown 2020; Lu 2021; DeepSeek-R1)
- **The out and the stop** — permission to say "I don't know" can reduce unsupported claims, because training rewards guessing; numbered scope plus a stop rule bound the sprawl. (Anthropic docs; OpenAI 2025)

CHECK — How to read evidence claims

Every claim travels with its task, model, date, and limitation. Check the original publication when a specific claim matters to your decision.

Why each dial matters: the agentic criteria

The second map explains why an agentic mission brief needs operational requirements. Use it whenever a task introduces new tools, actions, or approval boundaries.

- **Posture and standing rules** — conduct that must survive hour three of an unsupervised run; rules live in re-read files, not fading context. (Anthropic harness guidance)
- **Done plus verifiable-by** — the loop needs an exit it can test: complete "only after end-to-end verification — not when the code is written." (Anthropic memory-tool guidance, verbatim)

- **Environment boundaries** — immutable inputs and versioned outputs engineer reversibility in before the first action; blast radius is a design choice. (Claude Code protected paths)
- **Least-privilege tools** — name what exists, forbid the rest; risk-rate each tool by write access, reversibility, and impact. (OpenAI practical guide; NCSC)
- **Input register and trust** — trust level dictates validation depth; UNKNOWN beats a plausible guess, which propagates through the whole run. (Pipeline and register discipline)
- **Plan steps and methods** — named inputs and outputs per step keep intermediates inspectable; boring methods re-run identically; math runs as code. (Anthropic chaining; Faith and Fate 2023)
- **Checks: anchor, hard, soft** — rules-based feedback is "the best form of feedback" an agent gets; one outside anchor checks a specific relationship; further failure modes need their own checks. (Anthropic Agent SDK)
- **Gates and autonomy** — approval sits where errors are cheapest: at the plan, before irreversible actions, at failure thresholds; autonomy is named by the role you keep. (OpenAI guide; Feng et al. 2025)
- **Logs and reproducibility** — structured updates and descriptive commits make a run auditable; same inputs producing same outputs is engineered, not hoped. (Anthropic harness guidance)
- **Report shape and discipline** — a fixed status shape reads in thirty seconds; "done" means the check it ran, the output it got, the file re-opened. (SDK status enums; harness docs)

KEY — None of the blocks is bureaucracy

Each block of the mission brief exists because a documented failure taught someone to add it. Where a check feels awkward to write is where the process was fuzzy all along.

The evidence box set: works, myth, expired

The course sorts techniques into three verdicts: **works** (replicated, use it), **myth** (never survived replication, including some standard 2023 advice), and **expired** (was genuinely right, then the product absorbed it). This selection is compact; the full compendium is in [references/Elements_of_Prompting_Field_Guide.pdf](#) and the PLAYBOOK tab of [Course_Workbook.xlsx](#), researched as of September 2026. This field moves; keep the dates attached.

WORKS — Be specific: task, constraints, success criteria

The model fills every unstated need with the most plausible average guess, which matches your intent only about 41% of the time (Yang 2026).

WORKS — One tested, versioned template with delimiters

Formatting alone swings accuracy by up to 76 points; a tested template removes that lottery and makes runs comparable (Sclar, ICLR 2024; He 2024).

WORKS — Long inputs: documents at the top, question at the end

Attention concentrates at the edges of the context and thins in the middle; placement alone is worth up to about 30% on long documents (Liu 2023; Anthropic long-context guidance).

WORKS — Give it an out, in checkable shape, plus citations

Training rewards a confident guess over "I don't know"; an explicit UNKNOWN path cut unsupported answers from 66% to 44% (Omar 2025; OpenAI 2025).

WORKS — Self-check against named criteria, blinded from generation

Named criteria give the check something falsifiable to fail; fact scores jumped from 56 to 71 with planned verification (CoVe, ACL 2024).

WORKS — Ask the AI to improve your prompt (metaprompting)

Prompt wording is a search problem models solve better than people; optimizer-written prompts beat human ones (OPRO, ICLR 2024; GEPA 2026).

MYTH — "Tell it it's a genius and it gets smarter"

Personas move style, not competence: 162 expert personas tested, zero accuracy gain, and dumbed-down personas actively hurt (Zheng, EMNLP 2024; Wharton replication, Dec 2025).

MYTH — Tips, threats, "this is important to my career"

Effects wash out to an honest average of about 2.6%, null on modern models, while adding per-question noise; real stakes stated as factual context are fine (Salinas 2024).

MYTH — "Are you sure?" as verification

Pushback is a social signal, not a re-derivation: answers flip in 58% of cases, 14.7% of them from right to wrong (SycEval, AIES 2025; Huang, ICLR 2024).

MYTH — "Longer prompts are better"

The useful variable is information density, not length: reasoning accuracy fell from 0.92 to 0.68 with about 3,000 tokens of padding; detail helps, filler hurts (Levy et al., ACL 2024).

EXPIRED — "Let's think step by step"

Real in 2022, when it took one math benchmark from 10% to 41% (Kojima 2022); vendors then built the stepping in, so pick a reasoning model and set the effort dial. It remains legitimate on small, local, non-thinking models.

EXPIRED — Piles of ten-plus few-shot examples

In-context learning founded prompt engineering in 2020 (Brown 2020); instruction tuning absorbed most of it, and piles now degrade reasoning models (DeepSeek-R1 paper, 2025). Go zero-shot first, then 3-5 format-definers.

EXPIRED — Hand-run voting and micro-decomposed steps

Absorbed into test-time compute; over-prescribed steps now underperform outcome-first prompts (Wang, ICLR 2023; Anthropic 2026). Use effort settings; chain only where auditable intermediates are required.

EXPIRED — Carrying an old prompt stack onto each successive model

Scaffolds tuned to one model's quirks actively degrade its successors (GPT-5.5 guide, Apr 2026). Strip to the anatomy, re-test your gold cases, re-add only what earns its place.

KEY — The bias under the myths: the mirror

AI agrees with you about half again as often as a person would (+49% across 11 leading models; Stanford/CMU, Science 2026). So: never reveal your preferred answer, ask for the case against, paste your own draft as a colleague's, and treat agreement under pushback as noise.

Two honest edges. Politeness engineering is inconsistent in both directions: be normally civil, do not engineer tone. And "prompt engineering is dead" is a half-myth: the job title faded, but phrasing still swings results by double digits on frontier models.

Sources this training is built on

Every evidence claim in this course traces to a vendor document, a study, or a dated snapshot.

Vendor guidance (fetched August 2026)

- **Anthropic** — Prompting best practices; long-context; reduce-hallucinations; Building Effective Agents; Claude Code; context engineering.
- **OpenAI** — GPT-5 prompting guide; reasoning best practices; A Practical Guide to Building Agents; the Model Spec.
- **Google** — Gemini for Workspace guides (Persona-Task-Context-Format).
- **Microsoft** — Copilot guidance (Goal-Context-Source-Expectations); Copilot agents docs.

Research

- Brown et al. 2020 (few-shot); Wei et al. 2022 and Kojima et al. 2022 (chain-of-thought); Wang et al. 2023 (self-consistency).
- Sclar et al. 2024 (format brittleness); Zheng et al. 2024 (personas); Salinas and Morstatter 2024 (tips and perturbations).
- Cheng et al., Science 2026 (sycophancy); Sharma et al. 2023 (why sycophancy); Liu et al. 2023 (lost in the middle).
- OpenAI 2025 (why language models hallucinate); Sprague et al. 2025 (when chain-of-thought helps).

Landscape and governance

- Stanford HAI AI Index 2026; LMArena, Artificial Analysis, and SWE-bench (August 2026 snapshots).
- OWASP LLM Top 10 (2025) and Top 10 for Agentic Applications (2026); UK NCSC agentic-AI guidance; IMDA/CSA Singapore; Cisco AI Defense; Harmonic Security.
- Vendor launch posts and reporting (Reuters, CNBC, TechCrunch, Bloomberg) for all product dates.

Recommended reading: start practical, go deep

- **Practical start** — Mollick, Co-Intelligence (2024): the working-with-AI mindset book, read first. Kneusel, How AI Works (2024): the concepts of Part 1, gently and rigorously.
- **The craft** — Phoenix and Taylor, Prompt Engineering for Generative AI (O'Reilly 2024): the principles applied. Berryman and Ziegler, Prompt Engineering for LLMs (O'Reilly 2024): how prompts meet the model.
- **Going deep** — Huyen, AI Engineering (O'Reilly 2025): RAG, agents, evaluation. Alammam and Grootendorst, Hands-On Large Language Models (2024): visual internals, tokens to transformers.

TRY IT — The fastest teacher of all

A real task. Adapt a course example, run it, check the result, and save the improved prompt. Further reading is optional; one checked real task is the immediate goal.

The package

Everything the course produces lives in two folders. The deliverables/ folder holds the five artifacts you will open and reuse:

- **From_Prompts_to_Agents_Facilitated_60_Minute.pptx** — the facilitated deck, with full speaker notes.
- **Course_Workbook.xlsx** — the exercises: staged prompts, self-checks, the supplier dataset, and the PLAYBOOK tab.
- **Prompt_Template_Creator.html** — the interactive builder on the element taxonomy: pick an artifact, choose requirement types, assemble the prompt.
- **From_Prompts_to_Agents_Course.pdf** — this book: the course, readable with no session attended.
- **From_Prompts_to_Agents_Cheat_Sheet.pdf** — the one-page distillation for the desk.

Everything else sits under references/ (the build sources in src/ and the legacy prompt library sit beside the package root): the Elements_of_Prompting_Field_Guide.pdf with the full evidence compendium, the Prompt_Element_Taxonomy_Reference.pdf, the Prompt_Template_Configurator.xlsx, the facilitation plan, and exercise-data/ with the supplier materials — the clean CSV, the dashboard, the mock presentation, the email thread, the three quotation PDFs, the plain-language sources under references/exercise-data/plain-language/, and the instructor keys. The supplier scenario (Alpha Components, Bravo Plastics, Cardinal Metals) is entirely fictional: realistic data, no real company information.

CHECK — Leaving the appendix

You are done with this course when you can name the requirement types your next artifact needs, write them into a brief, and point at the check that would catch the most likely failure.